MAKERERE UNIVERSITY

P.O. Box 7062 Kampala Uganda
https://www.cs.mak.ac.ug

Telephone: 256-414-534560/1-9
E-mail: cs@cis.mak.ac.ug

COLLEGE OF COMPUTING AND INFORMATION SCIENCES

Traxis: Public Transport Coordination Application System

by

Group 7

Department of Computer Science
School of Computing & Informatics Technology

A Project Report Submitted to the School of Computing and Information Technology in Fulfilment of the requirements for the Award of the Degree of Bachelor of Science in Computer Science of Makerere University

Supervisor
Dr. Marriette Katarahweire

Department of Computer Science
School of Computing & Informatics Technology
kmarriette@cit.ac.ug Tel: +256414540628

November 2026

Declaration

contents We, Group 7 do hereby declare that this Project Proposal is original and has not been published and/or submitted for any other degree award to any other University before.

SNo	Names	Registration Number	Signature
1	Sunday Emmanuel Hezekiah	23/X/23054/PS	
2	Mulindwa Mark Myles	23/U/11949/PS	
3	Naluwujja Ashley Hope	23/U/14174/PS	
4	Kiggundu Sulaiman	22/U/21927/PS	

Date:

Approval

This Project Proposal has been submitted for examination with the approval of the following supervisor.

Signed:

Date:

Dr. Marriette Katarahweire
School of Computing & IT
College of Computing & IT
Makerere University
kmarriette@cit.ac.ug

Dedication

We dedicate this report to the Almighty God without whom, we could have done nothing. We further dedicate this report to our parents and guardians for the unceasing and selfless support throughout our stay in the University.

Acknowledgements

table of contents We would like to express our sincere gratitude to our project supervisor, Dr. Marriette Katarahweire, for her invaluable guidance and feedback during the preparation of this proposal. Her insights have been instrumental in shaping the direction of our research, and we look forward to her continued mentorship throughout the project's development.

We also extend our thanks in advance to the taxi drivers, conductors, and commuters in Kampala who will participate in our interviews and surveys. Their willingness to share their experiences and challenges will be fundamental to defining the requirements for the Traxis application.

Furthermore, we acknowledge the Department of Computer Science at Makerere University for providing the academic framework and resources that will enable this research.

Finally, we recognize the crucial role that open-source technologies and frameworks will play in the development of the Traxis system.

Abstract

Private shuttle transportation services in Uganda, particularly on intercity routes such as Kampala–Kabale, face significant challenges related to coordination, booking management, and operational efficiency. These services largely rely on manual processes, including phone-based bookings and informal passenger coordination, which result in inefficiencies such as unreliable seat allocation, unutilized capacity, and limited visibility for both operators and passengers.

This study addressed these challenges through the design and development of Traxis, a mobile application aimed at improving the organization and efficiency of private shuttle operations. The system was designed to support seat-based booking, real-time vehicle and occupancy tracking, and structured scheduling of shuttle services. It also incorporated features for managing passenger information, monitoring vehicle movement, and facilitating communication between operators and users.

Traxis adopted a dual-interface approach, providing functionality for both passengers and operators. Passengers were able to search for available routes, book specific seats, and receive real-time updates, while operators were able to monitor trips, track vehicle activity, and manage bookings through an administrative dashboard. The system design emphasized usability and accessibility to accommodate users with varying levels of digital literacy.

The project demonstrated the potential of mobile technology to improve coordination and resource utilization within privately operated shuttle services. By introducing structured booking, real-time tracking, and integrated operational management, Traxis provided a scalable and context-aware solution for enhancing efficiency, transparency, and reliability in Uganda's intercity transport sector.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Background to the Study

Public transportation remains a critical component of mobility in Uganda, particularly in rapidly growing urban centers such as Kampala [1]. While traditional taxi systems (matatus) serve a large portion of the population, there has been a growing demand for more reliable, comfortable, and organized transport alternatives [2]. Private shuttle services have emerged to fill this gap by offering scheduled trips, reduced overcrowding, and improved travel experiences. Despite these advantages, many of these services still operate with limited technological support [3].

Private shuttle operators often rely on manual processes such as phone calls and informal record-keeping to manage bookings, communicate with passengers, and coordinate trips. This approach presents several challenges, including double bookings, customer no-shows, difficulty in tracking seat availability, and inefficiencies in managing routes and schedules [4]. Additionally, customers may face challenges in accessing these services, as they are often only known through referrals or local advertisements, limiting their reach and accessibility.

The lack of a centralized digital platform also affects courier services offered by these operators. Package delivery coordination, pricing, and tracking are typically handled manually, making it difficult to provide real-time updates and efficient service [5]. This results in frequent customer inquiries, operational delays, and missed opportunities for optimizing vehicle usage.

With the increasing penetration of smartphones and mobile internet in Uganda [6], there is a significant opportunity to enhance the operations of private transport services through technology. Mobile applications that support features such as seat booking, digital payments, real-time tracking, and automated communication can greatly improve both customer experience and operational efficiency [7], [8].

This project proposes the development of a mobile application, Traxis, aimed at supporting private shuttle and courier service providers. The system will enable users to easily discover available services, book seats, select preferred seating positions, and track journeys or deliveries in real time. For operators, the application will provide tools for managing bookings, monitoring vehicle movement, tracking revenue, and optimizing capacity utilization. By introducing a structured and user-friendly digital platform, the system aims to

improve reliability, accessibility, and overall service delivery within the private transport sector in Uganda.

1.2 Problem Statement

Private shuttle transport services in Uganda, particularly in urban centers like Kampala, face significant challenges related to coordination and service management despite offering a more reliable alternative to traditional public transport. These challenges result in inefficiencies such as limited service visibility, difficulties in accessing booking information, customer no-shows, and underutilization of available seating capacity. The reliance on manual processes, such as phone calls and informal scheduling, further complicates operations and limits the ability of service providers to effectively manage passenger demand.

The absence of a structured, technology-enabled platform to support real-time booking, seat allocation, and service coordination exacerbates these issues. Consequently, customers often experience inconvenience and uncertainty when trying to access these services, while operators face operational inefficiencies that can lead to revenue losses and reduced service quality. Without an accessible and user-friendly digital solution, these challenges will persist, highlighting the need for an application such as Traxis to enhance communication, improve transparency, and optimize the management of private shuttle transport services in Uganda.

1.3 Objectives

1.3.1 Main Objective

The main objective of this project is to design and develop a mobile application (Traxis) that facilitates efficient management of private shuttle and courier transport services. The system will incorporate features such as real-time vehicle tracking, digital seat booking, and passenger scheduling for pickups. This initiative aims to address key operational inefficiencies by improving coordination between service providers and customers, reducing booking-related challenges such as no-shows, and optimizing seat utilization. Ultimately, the application seeks to enhance the reliability, accessibility, and overall efficiency of private transport services.

1.3.2 Specific Objectives

We aim to achieve the following specific objectives with the project:

i. To gather comprehensive data on passenger requirements, operational challenges faced by private shuttle and courier service providers, and the functionalities of existing transport management systems.

ii. To synthesize and analyze the acquired data to define a comprehensive set of system requirements for the Traxis application.

iii. To design and implement the proposed mobile application with all core feautures.

iv. To test and validate the performance and overall usability of the deployed system against the defined requirements and user expectations.

1.4 Scope of the Project

The scope of the Traxis project encompassed the development of a mobile application designed to improve the organization and reliability of private shuttle and courier transport services in Uganda, with initial focus on urban- to-upcountry routes such as Kampala to Kabale. The application provides core functionalities including real-time vehicle tracking, seat booking and allocation, and passenger scheduling for pickups. It also supports basic courier service management, including request placement and delivery tracking.

The system excluded other forms of transport such as boda bodas and standard public taxi (matatu) services in its initial phase in order to maintain a clear focus and ensure manageable system development within the project timeframe.

The system caters primarily to private shuttle operators and passengers who own smartphones and had access to mobile data, with an emphasis on ease of use to accommodate a range of digital literacy levels. The application enables shuttle operators to manage bookings and identify passenger demand hotspots, while allowing passengers to make informed travel decisions based on seat availability and scheduled departures.

Additionally, the project scope covered backend development to support real-time data processing, basic communication features for operator-passenger interactions, and user feedback mechanisms to allow iterative improvement. The project did not initially incorporate advanced features such as extensive multi- city coverage; however, it was designed to allow potential future scaling based on user needs and technical feasibility.

1.5 Significance of the Study

The Traxis project holds a substantial significance for multiple stakeholders within Uganda's private shuttle transport ecosystem and represents a meaningful step toward digital transformation in the informal transport sector. This study was justified by several critical factors:

- **To Passengers:** The application addresses customer challenges experienced in private shuttle transport services by enabling structured seat booking, real-time vehicle tracking, and scheduled pickups. It reduces uncertainty in travel arrangements by allowing users such as passengers to view available seats, select preferred seating, and receive instant booking confirmations. The system also improves coordination during journeys

by updating seat availability in real time when passengers disembark along the route, allowing better utilization of vehicle capacity.

- **For courier services:** the application supports convenient package pickup requests, real-time tracking of deliveries, and automated status notifications to both senders and recipients. This reduces the need for manual follow-ups and improves transparency in goods transportation.
- **To private shuttle operators:** The application offers private shuttle operators improved operational efficiency through enhanced visibility of passenger demand, better booking coordination, and reduced empty return trips. It enables optimized seat utilization, minimizes losses from unfilled return journeys, and improves daily earnings. The system also helps manage challenges such as customer no- shows and manual booking overloads through structured digital confirmations. In addition, the application provides real-time vehicle monitoring, allowing operators to track whether a shuttle is moving, stopped, or delayed along its route. This improves operational control, enhances accountability for drivers, and enables better communication with customers regarding trip progress and estimated arrival times.
- **To Transportation System Development:** The project contributes to the gradual digital transformation and improved organization of Uganda's private shuttle transport sector. By introducing structured booking, tracking, and coordination mechanisms, Traxis supports the development of a more reliable and efficient transport ecosystem.
- **To Technological Adoption:** This study demonstrates the feasibility of implementing mobile-based transport solutions in environments with varying levels of digital literacy. By incorporating phone-number-based access, simple interfaces, and optional customer support via calls, the system reflects appropriate technology design suited to the Ugandan context.
- **To Future Research and Development:** The project establishes a foundation for further innovation in private transport and logistics systems, including expansion to multiple routes, improved revenue analytics, integrated communication systems, and enhanced logistics tracking for both passengers and goods.

1.6 Justification

The justification for undertaking the Traxis project stems from the urgent need to address coordination and operational inefficiencies within Uganda's private shuttle and courier transport services. The current largely manual system, which relies on phone calls, informal scheduling, and limited digital support, results in challenges such as booking inconsistencies, customer no-shows, underutilized vehicle capacity, and lost revenue opportunities for service providers. These inefficiencies reduce overall service reliability and limit the growth potential of the sector.

At the same time, the increasing penetration of mobile technology and internet access in Uganda presents a timely opportunity to implement a digital solution tailored to this context. The Traxis application is therefore justified as a practical and context-aware intervention designed to bridge these operational gaps. By leveraging widely available technologies such as mobile data, GPS tracking, and mobile money integration, the project aims to improve booking coordination, enhance transparency in service delivery, and support real-time communication between operators and customers.

Ultimately, the system is intended to deliver immediate and practical benefits to passengers and private shuttle operators while also improving the efficiency of courier service operations and contributing to the gradual digital transformation of the transport sector in Uganda.

Chapter 2

Literature Review

In Uganda's capital, Kampala, private shuttle and intercity transport services operate alongside other informal transport systems, serving passengers traveling between urban and upcountry destinations [9], [10]. While these shuttle services are generally more comfortable and reliable than traditional public taxis, they still operate within a largely manual and semi-organized framework that presents challenges to both passengers and service providers [11], [12], [13].

In many cases, private shuttle operators rely on phone calls, referrals, and informal scheduling to manage bookings and coordinate trips [11]. Departures are typically fixed within specific time windows, but seat allocation and passenger confirmations are often not managed through a centralized system. This can result in inefficiencies such as seat underutilization, customer no-shows, and difficulty in managing last- minute bookings [12], [4].

From the passenger perspective, accessing these services can also be challenging, as information about available routes, departure times, and seat availability is not always easily accessible in real time [14]. In addition, courier services offered by shuttle operators are often managed manually, with limited tracking capabilities and reliance on direct communication between senders and operators [4].

These limitations highlight gaps in the coordination and digital integration of private shuttle and courier services. Existing literature on transport systems suggests that mobile-based platforms with real-time booking, tracking, and communication features can significantly improve operational efficiency and service reliability, particularly in environments where transport systems remain semi-formal and manually coordinated.

2.1 Technologies Enabling Transport Digitization

In recent years, the rise of mobile technology has revolutionized the development of urban transport systems in many different countries [15] [16]. Mobile phones, smartphones in particular have become increasingly accessible, offering options for users across different income levels [17]. Brands such as Tecno, Infinix, Itel, and Samsung's budget-friendly A-series provide affordable devices for a broad population, while high-end smartphones from Apple, Samsung, and Google Pixel cater to users seeking premium technology [18] [19] [20]. In ad-

dition to smartphones, wearable devices such as smartwatches also support the development of transport systems [21]. They also offer both affordable and high-end options. These mobile devices offer a platform for lightweight ride hailing applications that can communicate real time information on vehicle location and availability [22]. They provide GPS support that allows vehicle tracking and route optimization [23]. They also accommodate internet connectivity to allow users to access various ride hailing services [24].

In addition, cashless payment systems enabled by technologies like Near Field Communication (NFC) and Radio-Frequency Identification (RFID, allow commuters to pay fares using their debit or credit cards, mobile wallets or even smart cards [25]. Examples include contactless cards like Visa and Mastercard, mobile payment apps like Apple Pay, Google Pay, MTN Mobile Money and Airtel Money and on-board sales units that accept card payments. These cashless systems reduce transaction times, improve convenience, and minimise the need to handle physical money [26]. Modern ride-hailing applications typically offer multiple payment options, allowing users to select their preferred method [27]. For instance, Uber supports payment via credit/debit cards, and in Uganda users can also pay in cash by selecting the "cash" option in the app [28]. SafeBoda uses its in app Wallet, which can be topped up via mobile money (e.g. MTN, Airtel) or credit or debit cards [29]. Bolt allows in app payments via credit or debit cards or mobile wallets [30].

Improvements in mobile internet connectivity have significantly enhanced global access to ride-hailing and transport-digitization platforms [31]. The expansion of 3G and 4G networks and the gradual introduction of 5G by telecom providers such as MTN, AT and T, Verizon, and Airtel has increased mobile internet speeds and network reliability [32]. High speed connectivity is essential for the operation of services like Uber and Lyft, enabling instant drivers and passengers to be matched instantly, real-time GPS-based vehicle tracking, and smooth processing of digital payments [27]. Navigation platforms such as Google Maps and Waze rely on continuous mobile data connections to deliver live traffic updates, suggest alternative routes, and provide precise turn-by-turn navigation, ultimately reducing travel times and fuel consumption for drivers [33] [34].

Consistent mobile internet access plays an important role in supporting modern transport information systems by enabling users to access real-time travel information such as schedules, service updates, and digital ticketing options [35] [36]. In addition, improved mobile connectivity has enabled transport and logistics operators to adopt systems that support remote monitoring of vehicle location, speed, and operational status. Commercial transport operators increasingly use such technologies to enhance operational efficiency, improve service coordination, and support better decision-making. These capabilities also contribute to improved accountability and more structured transport management practices within transport and logistics networks [37] [38].

Cloud-based systems are a key enabler in the digitization of the global transport sector, providing scalable and flexible infrastructure for managing real-time data, optimizing operations, and improving user experience [39]. In modern transport and logistics systems, cloud computing supports the processing of large volumes of live data, including vehicle location through GPS, traffic conditions, booking activity, and system interactions [40]. This capability enables real-time vehicle monitoring, improved coordination between users and operators, and more accurate travel information such as estimated arrival times and service availability. In addition, cloud-based architectures support efficient data sharing between

different system components, making them suitable for transport and logistics applications that require continuous updates and multi-user access.

Demand for digital transport services often fluctuates depending on peak travel periods, holidays, and special events. Cloud-based platforms address this variability by providing automatic scalability, allowing computing resources to expand or reduce based on system demand [41] [42]. This ensures consistent system performance, prevents overload, and reduces the need for extensive physical infrastructure.

In addition, cloud-enabled transport systems can support intelligent data processing capabilities such as route optimization by utilizing real-time traffic information and other contextual data. This helps improve travel efficiency by reducing delays and enhancing operational planning [43]. Furthermore, automated allocation mechanisms within such systems can streamline operations by matching available transport resources with incoming service requests, improving response time and overall coordination [44].

Cloud-based analytics platforms enable real-time analysis of demand and supply data, allowing transport systems to respond dynamically to changing user requirements. This capability supports operational adjustments that improve service allocation efficiency and enhance overall system performance [45]. In transport and logistics contexts, such data-driven approaches can help improve resource distribution by identifying areas of high demand and optimizing the utilization of available vehicles.

In addition, cloud environments support continuous integration and deployment practices, enabling rapid feature updates and system improvements without significant downtime [46]. This ensures that transport applications remain adaptable, maintainable, and capable of evolving in response to user needs and operational challenges.

Cloud-based platformssupport vehicle health tracking, contributing to better maintenance planning and regulatory compliance [47]. Additionally, cloud providers offer robust security features such as encryption, intrusion detection, and multi-factor authentication to protect sensitive user and payment data [48]. They also assist transport platforms in complying with international data protection standards, such as the General Data Protection Regulation (GDPR) [49].

2.2 Existing Private Ride-hailing Platforms

Having established the technological foundations that enable digitized transport solutions, it is instructive to examine how these technologies have been applied in practice through local and international platforms operating in Uganda. For these digital platforms to succeed in Uganda, they must navigate a complex interplay of opportunities and challenges that differ markedly from more developed markets. Below are some explorations into existing research and case studies on transport and courier apps operating within Uganda, shedding light on both their transformative potential and the persistent hurdles they face. Through this examination, the fundamentals that underpin effective app-based public transport services in Uganda become clearer, guiding the development objectives for Traxis.

2.2.1 Uber

Uber technologies, is a global transportation company with its headquarters in San Francisco, California, USA. It develops, markets and operates the Uber car transportation and food delivery mobile applications [50]. Uber drivers use their own cars or they can rent a car to drive with Uber after passing a criminal background check by Interpol [51]. The Uber application software requires drivers to have smartphones and users must have access to either a smartphone or a mobile website. A user provides his or her destination and then requests a ride; Uber performs a geo-location search for the nearest driver available and lets a user confirm the request indicating the fare to be paid. When a driver accepts the request, a notification is received and a user can then monitor the car movement on the maps fragment of the application. Details like name of driver, vehicle registration, make of vehicle and approximated arrival time is provided to the user [52].

Uber's introduction to Kampala marked a significant innovation in the urban transport sector by formalizing the taxi experience through smartphone technology [53]. Before Uber, commuting largely depended on informal taxis with limited regulation and inconsistent service quality. Uber's model introduced on-demand rides, GPS-based tracking, and cashless payments, dramatically enhancing convenience and transparency [53].

However, the integration of Uber into Uganda's transport ecosystem faced several challenges, including regulatory resistance and limited adoption beyond higher- income urban users due to affordability constraints and smartphone accessibility [53] [54].

Uber's operational model differs significantly from the requirements addressed by Traxis. While it facilitates point-to-point transport for individual users [53], Traxis is designed to support route-based travel where multiple passengers share a vehicle along predefined intercity routes. In such systems, passengers may board and disembark at different points during the journey, resulting in continuously changing seat availability. Unlike Uber, Traxis is designed to support mid-route passenger coordination, where a seat freed at one point along the journey can be reassigned to another passenger further along the route. This enables better seat utilization and reduces revenue loss for operators.

In addition, pricing models differ significantly. Uber fares are calculated dynamically based on distance and time, which can make long-distance travel costly [53] [54], [55]. In contrast, Traxis supports fixed and more predictable pricing structures for specific routes, making it more suitable and accessible for intercity travel.

2.2.2 Safeboda

SafeBoda is a motorcycle ride-hailing company headquartered in Kampala, Uganda, operating across several cities in East and West Africa. It develops, markets, and operates a mobile application that connects passengers to trained motorcycle taxi (boda boda) riders for urban transportation and delivery services [56].

SafeBoda riders use their own motorcycles and must undergo a rigorous onboarding process that includes background checks, road safety training, and customer care training before being allowed onto the platform. Riders are also provided with helmets and reflective jackets to enhance safety for both themselves and passengers [57].

The SafeBoda application requires riders to have smartphones, while users must have

access to either a smartphone or, in some cases, USSD/mobile-based alternatives. A user inputs their pickup location and destination, then requests a ride through the app. The system performs a geo-location search to identify the nearest available rider and provides an estimated fare before the ride is confirmed . For courier services, users can request a rider to pick up and deliver parcels to a specified destination. The system performs a geo-location search to identify the nearest available rider and provides an estimated fare before the request is confirmed [58].

Once a rider accepts the request, the user receives a notification and can track the rider's movement in real time through the map interface. The application displays key details such as the rider's name, profile photo, motorcycle registration number, and estimated time of arrival [59].

After the trip, both the rider and the passenger can rate each other on a scale of 1 to 5 based on factors such as safety, professionalism, and overall experience. This rating system helps maintain service quality and accountability.

In Uganda, SafeBoda supports both cash and cashless payments. Users can pay using mobile money services integrated within the app or opt to pay cash at the end of the trip, making the service accessible to a wider population [60].

While SafeBoda is generally more affordable than normal boda bodas [61], its pricing can still be costly for the average commuter, particularly over long distances where per-kilometre charges accumulate [62]. While this may be suitable for short urban trips, it becomes increasingly expensive and less predictable over longer distances. For intercity travel, such as journeys from Kampala to Kabale, fare estimates provided at the beginning of a trip may differ from the final cost due to factors such as traffic conditions, route changes, or delays. This reduces cost predictability for users and limits affordability for long-distance travel. In contrast, Traxis is designed to support fixed, route-based pricing models commonly used by private shuttle operators. These pricing structures are predetermined for specific routes, making them more affordable and predictable for passengers.

Moreover, SafeBoda operate on a point-to-point transport model, where a vehicle is assigned to a single trip for a specific passenger [63]. This model does not support shared, route-based transport where multiple passengers can board and disembark at different points along a journey. In contrast, Traxis is designed to support a route-based shuttle system in which passengers may disembark at intermediate locations, and their seats can be reassigned to other passengers further along the route. This dynamic seat management approach improves vehicle utilization and reduces revenue loss for operators while maintaining affordability for passengers.

2.2.3 Taxify

Taxify is an international transportation network company, founded and headquartered in Tallinn, Estonia. The application enables people to request a ride-hailing from their smartphones. Once the rider has conrmed the pickup location, requested a ride and the driver has accepted. The rider can see the drivers details and both the driver and rider rate each other after the ride. Taxify drivers undergo a criminal background check and in person training [64]. Taxify (now bolt) leverages Uganda's mobile money ecosystems like M-Pesa to facilitate digital payments and attract drivers by offering lower commissions than com-

petitors [65] [66]. Its market entry disrupted existing ride-hailing dynamics, providing an affordable alternative and extending services to less affluent users. The transition to the Bolt brand signaled a broader vision that includes multi-modal mobility options including e-scooters, reflecting shifting urban transport preferences. Bolt's strategic emphasis on safety features, local partnerships, and customer education has enhanced user and driver retention. This case highlights the critical role of financial inclusion, regulatory alignment, and diversified service offerings for ride-hailing success in Uganda's evolving urban transport landscape [67].

Bolt operates primarily as a private ride-hailing platform where a single booking typically reserves the entire vehicle for one user and their companions. While the platform allows features such as upto three stops within a trip and account-based ride management, it does not support independent seat-based booking by multiple strangers within the same vehicle [68].

As a result, any form of shared travel on Bolt is coordinated through a single booking entity rather than through a structured system where individual passengers independently reserve seats within a shared vehicle. This means that the ride is pre-organized under one request, and additional passengers are included only through the original booking structure rather than through an open seat allocation system.

In contrast, private shuttle systems operate on a seat-based model where multiple unrelated passengers independently book seats within a shared vehicle operating on a fixed route. This creates a structured coordination system that is fundamentally different from Bolt's booking model.

Traxis is designed to support this shuttle-based structure by enabling independent seat booking, real-time seat availability updates, and coordinated passenger allocation within a single scheduled journey.

2.2.4 Lyft

Lyft, used in the USA and Canada, is a ride-hailing app like Uber [69]. Lyft offers a Shared ride option designed to facilitate carpooling by matching passengers traveling along similar routes. In this service, each booking is typically limited to one or two passengers per request. This approach enables cost savings through shared travel [70]. In contrast, a standard Lyft ride provides a private trip for a single party of up to four passengers, ensuring exclusive use of the vehicle. For larger groups, Lyft XL is available, accommodating up to six passengers in a single booking [71]. These service tiers allow Lyft to cater to different user needs, balancing affordability, convenience, and capacity. Lyft also provides courier and delivery services through its "Lyft Delivery" offering, which enables drivers to transport a range of items including groceries, retail goods, auto parts, and medical supplies [72], [73], [74]. Unlike fully consumer-facing delivery platforms, this service primarily operates through partnerships with businesses rather than direct user requests within the standard Lyft application. Deliveries are typically initiated via integrated platforms or third-party partners, such as Olo, allowing restaurants and other organizations to request on-demand, same-day delivery service [75]s. Lyft Delivery is therefore largely structured around business-to-business (B2B) and business-to-consumer (B2C) models, rather than person-to-person (P2P) courier services [76]. The service is available in select cities across the United States and parts of Canada, and focuses

mainly on essential deliveries such as food, retail items, and pharmacy products.

Despite its diversification, Lyft's operational model remains centered on urban ride-hailing and on-demand delivery in highly structured digital environments. It does not support fixed-route, intercity shuttle transport systems where passengers independently book seats within a scheduled vehicle operating along a predefined route.

In contrast, Traxis is designed specifically for private shuttle operations in contexts such as Kampala–Kabale routes, where transport is organized around shared vehicles, fixed departure schedules, and seat-based allocation for independent passengers. Unlike Lyft's ride-matching or ride-sharing mechanisms, Traxis enables structured seat booking within a single scheduled journey, allowing coordinated management of occupancy for intercity travel.

Furthermore, while Lyft Delivery is largely platform- and partner-driven, Traxis integrates passenger transport and courier services within a single shuttle- based system, enabling direct coordination between operators and users for both seats and parcel movement within the same operational framework.

2.2.5 Moovit

Moovit is primarily a comprehensive public transit navigation and Mobility as a Service (MaaS) app, not just a ride-hailing app like Uber. It acts as a trip planner that combines public transit (buses, trains, subways), bikes, scooters, and Moovit also integrates with ride-hailing services. It aggregates data from multiple transport providers, including buses, trains, and informal transit systems, to assist users in planning efficient journeys. The application utilizes real-time data, GPS tracking, and user-generated updates to provide accurate arrival times, route options, and service alerts. Users input their origin and destination, and the system generates optimal routes by combining different modes of transport, including walking segments where necessary [77].

Moovit also supports features such as live directions, step-by-step navigation, and service disruption notifications, enhancing the overall commuting experience [78]. In addition, the platform contributes to urban mobility planning by providing data analytics tools to cities and transport authorities [77].

However, Moovit functions primarily as an information and navigation system rather than an operational transport coordination platform. It assists users in selecting routes but does not directly manage vehicle occupancy, seat allocation, or booking within shared transport vehicles.

In contrast, Traxis is designed as an operational coordination system for private shuttle services operating on fixed intercity routes such as Kampala–Kabale. Instead of only providing route information, Traxis enables direct seat booking, real-time seat availability updates, and passenger allocation within specific shuttle vehicles. It also supports operator-level coordination, including monitoring of vehicle status and passenger demand. Furthermore, while Moovit focuses on aggregating transport data across multiple providers.

2.2.6 Transit App

The Transit App is a mobility application designed to help users plan, navigate, and manage public transportation journeys in real time. The application enables users to enter a

destination and view available transport options, including buses, trains, and other nearby mobility services, along with estimated departure times and route alternatives. Once a trip is selected, the app provides step-by-step navigation guidance, helping users move through each stage of their journey from origin to destination [79] [80].

A key feature of the Transit App is its real-time trip planning capability, which allows users to compare multiple route options based on travel time, convenience, and service availability. The app also supports live directions through its "GO" feature, which provides continuous guidance during travel, including notifications for when to leave, when to transfer, and when to alight from a vehicle. This ensures that users receive timely alerts throughout their journey without needing to constantly monitor the app [80].

Additionally, the Transit App provides service alert notifications, which inform users about disruptions such as delays, route changes, or cancellations. Users can also access nearby departures from their current location, improving convenience when planning spontaneous trips. In some cities, the app integrates additional mobility services such as ride-hailing, bikeshare, and e-scooters, offering a more comprehensive multimodal transport solution [80].

However, the Transit App functions primarily as an informational and navigation tool rather than a transport coordination system. It does not manage vehicle occupancy, seat allocation, or booking within specific transport services.

In contrast, Traxis is designed as an operational coordination system for private shuttle services operating on fixed intercity routes such as Kampala–Kabale. Instead of only providing route guidance, Traxis enables users to directly book seats, view real-time seat availability, and coordinate travel within a specific shuttle vehicle.

Furthermore, while the Transit App aggregates information across multiple transport providers to assist in journey planning, Traxis focuses on managing a specific transport operator's services, integrating seat-based passenger booking, real-time vehicle tracking, and courier coordination within a single structured system.

O

2.2.7 SWVL

Swvl is a technology-driven mass transit platform that aims to transform urban transportation by providing efficient, affordable, and accessible shared mobility solutions. Operating across multiple cities in the Middle East, Africa, Asia, and Latin America, Swvl leverages data-driven systems to improve public transport efficiency and reduce reliance on private vehicles. The platform addresses key urban mobility challenges such as traffic congestion, inefficient routing, limited transport accessibility, and safety concerns by introducing optimized, demand-responsive transit solutions [81].

Swvl operates through a digital ecosystem that connects passengers to shared buses and vehicles via mobile applications. Using real-time data, adaptive routing algorithms, and predictive demand models, the system designs efficient travel routes, improves vehicle occupancy rates, and reduces travel time. It also introduces virtual stops and flexible routing to enhance first- and last- mile connectivity, making public transport more accessible to users in both densely populated and underserved areas. Additionally, Swvl supports use cases

such as corporate employee transportation, university transit, and city-wide public mobility systems [82].

Beyond improving efficiency, Swvl emphasizes sustainability by reducing traffic congestion and lowering carbon emissions through shared rides and optimized fleet utilization. The platform integrates tools for fleet operators, drivers, and administrators, enabling real-time monitoring, scheduling, and service optimization. Overall, Swvl functions as a mobility-as-a-service (MaaS) platform designed to modernize traditional public transport systems and create safer, more reliable, and more sustainable urban mobility networks.

However, Swvl is primarily designed for structured mass transit systems and operates at a system-wide optimization level, rather than focusing on individual private shuttle operators managing specific intercity routes. Its model assumes centralized fleet management and algorithmic route planning across large-scale transport networks.

In contrast, Traxis is designed for privately operated shuttle services such as Kampala–Kabale routes, where transport is organized around fixed schedules, seat-based booking, and operator-controlled vehicles. Instead of dynamically redesigning routes, Traxis focuses on coordinating passenger bookings, managing seat occupancy in real time, and improving operational visibility for individual shuttle operators.

Furthermore, while Swvl optimizes transport at a network level using predictive algorithms, Traxis operates at the service-provider level, enabling direct coordination between passengers and a specific shuttle operator, including features such as seat allocation, real-time vehicle tracking, and integrated courier coordination.

2.2.8 Via

Via is a mobility-as-a-service (MaaS) platform that provides on-demand shared transportation solutions designed to improve urban transit efficiency [83]. The service allows users to book rides through a mobile application by entering their pickup location and destination. Once a request is made, the system matches passengers traveling in similar directions and assigns them to shared vehicles, reducing travel costs and improving vehicle occupancy [84].

A key feature of the Via App is the use of designated "virtual bus stops," where passengers are directed to nearby pickup points instead of exact door-to-door collection [84]. This approach minimizes route deviations, reduces travel time, and increases overall system efficiency. The platform also provides real-time tracking, estimated arrival times, and live trip updates to enhance user convenience and transparency.

Via's operations are powered by a proprietary algorithm known as "ViaAlgo," which analyzes traffic conditions, passenger demand, and vehicle locations to optimize routing and minimize waiting times. Unlike traditional ride-hailing services, Via primarily collaborates with public transport agencies and municipalities to complement existing transit systems, offering a flexible middle-ground between fixed-route buses and private taxis. Drivers receive turn- by-turn navigation and optimized pickup instructions through a dedicated driver application [84].

owever, Via operates primarily as an algorithm-driven shared mobility system designed for dynamic ride pooling and demand-responsive transit. It does not focus on operator-managed private shuttle services where fixed routes, scheduled departures, and structured seat allocation are central to operations.

In contrast, Traxis is designed for privately operated shuttle services such as Kampala–Kabale routes, where transport is organized around predefined schedules, seat-based booking, and operator-controlled vehicles. Rather than dynamically generating ride pools, Traxis provides structured coordination of passengers within a specific shuttle, ensuring clear seat allocation and operational control for the service provider.

Furthermore, while Via emphasizes algorithmic matching and virtual stop optimization at a system level, Traxis focuses on operator-level management, including real-time seat occupancy tracking, passenger booking per seat, vehicle monitoring, and integrated courier coordination within the same shuttle operation.

2.2.9 Grab

Grab is a multi-service mobility and digital platform operating across several Southeast Asian countries, providing ride-hailing, food delivery, and financial services through a unified mobile application. The platform connects users with nearby drivers by using real-time geo-location technology to identify the closest available driver. Once a ride is requested, the application displays essential driver and vehicle information, including the driver's photo, vehicle details, and license plate number, to ensure safety and easy identification [85].

The Grab application offers flexibility by allowing users to either request immediate pick-ups or schedule rides in advance depending on their travel needs. This enhances convenience and supports both planned and on-demand transportation requirements. In addition, the platform includes an in-app messaging feature that enables communication between drivers and passengers, with built-in translation support to facilitate interaction across different languages [86].

However, Grab operates primarily as a super-app focused on individual ride- hailing and on-demand service delivery across multiple sectors. While it integrates several services within one platform, its transport model remains centered on point-to-point trips between a passenger and a driver, rather than structured coordination of shared, route-based transport systems.

In contrast, Traxis is designed specifically for private shuttle operations such as Kampala–Kabale routes, where multiple passengers share a scheduled vehicle operating along a fixed intercity route. Instead of focusing on individual ride matching, Traxis enables structured seat-based booking, real-time seat occupancy tracking, and coordinated passenger allocation within a single transport service.

Furthermore, while Grab integrates multiple consumer services within a super-app ecosystem, Traxis focuses narrowly on optimizing shuttle operations by combining passenger transport and courier coordination within a single structured system designed for operator-managed, route-based mobility.

2.2.10 Comparative analysis table of the different ride-hailing platforms

The table below provides a summary of the features of the different platforms discussed in the sections above.

Table 2.1: Comparative Analysis of Transportation Platform Features

Feature	Transport Model	Booking structure	Route structure	Seat management	Operational coordination	Pricing model	Courier integration
Traxis	Fixed-route private shuttle coordination	Independent seat- based booking	Fixed intercity routes (e.g., Kampala–Kabale)	Real-time seat allocation and tracking	Operator-level shuttle management	Fixed fare per seat and courier pricing	Fully integrated passenger and parcel system
Uber	Point-to-point rides or navigation support	Whole ride per request or trip planning only	Dynamic or user-defined	PNot exposed to users	Driver-passenger matching or route guidance	Dynamic or distance-based	Separate or limited
Safeboda	Point-to-point rides or navigation support	Whole ride per request or trip planning only	Dynamic or user-defined	PNot exposed to users	Driver-passenger matching or route guidance	Dynamic or distance-based	Separate or limited
Taxify (Bolt)	Point-to-point rides or navigation support	Whole ride per request or trip planning only	Dynamic or user-defined	PNot exposed to users	Driver-passenger matching or route guidance	Dynamic or distance-based	Separate or limited
Lyft	Point-to-point rides or navigation support	Whole ride per request or trip planning only	Dynamic or user-defined	PNot exposed to users	Driver-passenger matching or route guidance	Dynamic or distance-based	Separate or limited
Moovit	Point-to-point rides or navigation support	Whole ride per request or trip planning only	Dynamic or user-defined	PNot exposed to users	Driver-passenger matching or route guidance	Dynamic or distance-based	Separate or limited

Continued on next page

Table 2.1: Comparative Analysis of Transportation Platform Features (Continued)

Feature	Transport Model	Booking structure	Route structure	Seat management	Operational coordination	Pricing model	Courier integration
Transit App	Point-to-point rides or navigation support	Whole ride per request or trip planning only	Dynamic or user-defined	PNot exposed to users	Driver-passenger matching or route guidance	Dynamic or distance-based	Separate or limited
SWVL	Shared mass transit (bus-like systems)	Grouped/shared ride matching	Algorithm-optimized or flexible routes	Limited grouping, not seat-by-seat control	System-level fleet optimization	Variable/shared fares	Not core functionality
Via	Shared mass transit (bus-like systems)	Grouped/shared ride matching	Algorithm-optimized or flexible routes	Limited grouping, not seat-by-seat control	System-level fleet optimization	Variable/shared fares	Not core functionality
Grab	Point-to-point rides or navigation support	Whole ride per request or trip planning only	Dynamic or user-defined	PNot exposed to users	Driver-passenger matching or route guidance	Dynamic or distance-based	Separate or limited

2.3 Conclusion

In summary, the growth of ride-hailing and digital mobility platforms has marked a significant transformation in urban transportation systems globally and within Uganda. International platforms such as Uber have introduced digital convenience through features such as GPS tracking, cashless payments, and formalized booking systems, helping to modernize urban taxi experiences. However, their adoption in Uganda remains largely concentrated among higher-income urban users, limiting their inclusivity within the broader transport population.

Locally adapted platforms such as SafeBoda have demonstrated the importance of contextual design by integrating mobile payments, safety training, and affordability into their service model, ensuring wider usability within Uganda's transport ecosystem. Similarly,

platforms like Bolt have expanded access through competitive pricing and mobile money integration, further contributing to the evolution of app-based mobility services in the region.

Despite these advancements, existing platforms primarily focus on either private point-to-point transport or generalized ride-hailing services, with limited support for structured coordination of shared, fixed-route shuttle systems that dominate intercity travel in Uganda. This creates a gap in how seat-based occupancy, scheduled departures, and operator-managed shuttle services are coordinated digitally.

These observations highlight the need for a context-aware and system-specific solution tailored to Uganda's private shuttle transport sector. Traxis is therefore proposed as a digital platform designed to improve coordination, communication, and operational efficiency within fixed-route shuttle services through real-time seat management, structured booking, and integrated transport–courier functionality. i

Chapter 3

Methodology

3.1 Introduction

This chapter highlights the different techniques that where used to successfully complete the project. It provides a consise description for the data collection and analysis methods, system design and implementation methods, together with the system testing and validation techniques that were applied to make sure that Traxis satisfies and meets user requirements.

The key phases to be undertaken where:

- **Data Collection**: Comprehensive data on commuter needs, driver challenges, and existing platform functionalities was acquired through interviews, surveys and competitive analysis.
- **System Design and Implementation**: The overall architecture, UI/UX, and core features were then designed, developed and implemented for the full Traxis system.
- **Testing and Validation**: Comprehensive functional, performance, and usability testing were done to ensure the system meets the specified requirements and performs reliably under real-world conditions.

3.2 Data Collection

This phase focused on systematically identifying and analyzing both functional and non-functional requirements for Traxis. The primary objective was to deeply understand commuter pain points, driver operational challenges, and gaps in existing transport platforms in the private taxi ecosystem ensuring that the final system is grounded in real world needs rather than assumptions.

3.2.1 Data Collection Techniques

- **Semi-structured Interviews**: In-depth interviews with private shuttle coordinators, drivers, and frequent passengers were conducted to uncover coordination bottlenecks, safety concerns, trust issues, and desired digital interventions.
- **Questionnaires**: Targeted surveys were distributed to a broader passenger base (minimum target: 100 respondents) to quantify commuting patterns, smartphone penetration, willingness to adopt app- based solutions, and feature priorities.
- **Direct Observation**: Field observation at private shuttle stages was done to document real driver–passenger interactions, queue dynamics, information asymmetry, and operational inefficiencies that users may not articulate in interviews.
- **Competitive Analysis**: Evaluation of existing ride-hailing and transport apps operating in East Africa (e.g., Uber, Bolt, SafeBoda, digital matatu initiatives) was done to identify functional gaps and lessons applicable to the informal taxi sector.

3.2.2 Tools and materials

Data Collection Method	Tools and Materials
Questionnaires	Pens and printed questionnaires; Google Forms
Interviews	Interview guide; Pens and notebooks; Smartphones
Observations	Smartphones

Table 3.1: Tools and Materials Used for Data Collection

3.3 System Design and Implementation

To ensure effective and well-coordinated project management, the Agile software development methodology was be adopted throughout the project lifecycle. This approach contributed significantly to the success of the project by enabling iterative planning, continuous evaluation, and timely adjustments to project activities. By anticipating the need for flexibility in time management and task execution, the team was able to respond efficiently to changing requirements, minimize potential delays, and maintain steady progress toward the project objectives.

3.3.1 System Design

The system was architected around two primary entities to guarantee reliable communication, scalability, data consistency, and overall operational efficiency across the platform.

- **Clients**: A mobile application deployed on the smartphones of passengers and drivers. *The Application that functioned as client-side components, handling user interactions,*

location updates, and real-time requests while communicating securely with the backend services.

- **Server** (Single Source of Truth): A centralized, cloud- hosted backend system that acted as the authoritative source for all application data and system state. The server was responsible for coordinating passenger–driver matching, enforcing business rules, synchronizing data across all connected clients, and maintaining consistency in real-time operations. Additionally, it handled secure data storage, backups, authentication, logging, and fault tolerance, ensuring data integrity, system reliability, and recoverability in the event of failures or network disruptions.

Client Design

The client application was be developed using **Dart**, a modern, object- oriented programming language developed by Google and optimized for building fast, scalable user interfaces. Dart was used in combination with the **Flutter** framework, which enabled cross-platform application development from a single codebase. This allowed the application to run on both Android and iOS platforms while reducing development time, simplifying maintenance, and ensuring a consistent user experience across devices.

The client-side design adopted a modular architecture, where each functional feature was implemented as an independent component. Flutter's widget-based structure promoted reusability, scalability, and maintainability, while supporting responsive and adaptive user interfaces.

The client application integrated the following technologies:

- **Google Maps API** supported geolocation services, route visualization, distance calculations, and real-time location tracking. It provided a robust mapping platform that integrated seamlessly with mobile applications.

- **AWS Services** were used to implement transcription and other compute-intensive features. AWS (Amazon Web Services) offered scalable cloud-based computing resources, ensuring that tasks such as voice-to-text conversion were processed efficiently and reliably.

- **MCP (Model Control Protocol)** was employed to enable AI- powered features implemented in **Python**, a high-level programming language well-suited for machine learning, data processing, and AI integration. MCP allowed the client to interact with intelligent services through well-defined APIs.

Overall, the client design prioritized responsiveness, scalability, and seamless integration with backend services while maintaining a clear separation between presentation logic and data handling.

Server Design

The server-side component was deployed on a **cloud-based infrastructure** to ensure scalability, high availability, and fault tolerance. The backend was implemented using **Django**,

a high-level web framework built with **Python**, chosen for its simplicity, security, and rapid development capabilities.

Django served as the core framework responsible for processing client requests, handling business logic, managing API endpoints, and coordinating communication between clients and external services.

The server utilized **PostgreSQL**, an advanced open-source relational database, for all persistent data storage. PostgreSQL was known for its reliability, strong data integrity, support for complex queries, and transactional consistency, ensuring accurate management of user profiles, trip data, logs, and system metadata.

Gunicorn acted as the application server running the Django application in a production environment. Gunicorn is a Python WSGI (Web Server Gateway Interface) server that managed worker processes, handled HTTP requests, and ensured stable and efficient server performance.

The server also acted as the **single source of truth**, responsible for maintaining data consistency, synchronizing state across all clients, enforcing business rules, handling secure backups, and supporting recovery in the event of system failures.

Admin Portal Design

In addition to the client applications and server backend, the system included an **Admin Portal**, designed to provide administrators with a web-based interface for managing and monitoring all aspects of the platform. The Admin Portal was developed using **Vue.js**, a progressive JavaScript framework known for its simplicity, flexibility, and reactive component-based architecture. Vue.js enabled the development team to build a responsive, interactive, and maintainable frontend with efficient data binding and component reusability.

The Admin Portal allowed administrators to perform key functions, including:

- **User Management:** Adding, editing, and removing user accounts for passengers, drivers, and other system roles, while enforcing access controls and permissions.
- **Trip Monitoring:** Viewing ongoing trips, trip history, and statistics to ensure operational efficiency and service quality.
- **System Analytics:** Accessing dashboards that visualized critical metrics such as ride demand, driver availability, user activity, and AI feature performance.
- **Notifications and Alerts:** Sending announcements, system alerts, or notifications to users through integrated backend services.
- **Configuration Management:** Managing system parameters, feature toggles, and backend integrations, allowing dynamic adjustment of the platform without downtime.

The Admin Portal interacted with the centralized server through secure API endpoints, ensuring that all administrative actions were synchronized with the server's **single source of truth**. This integration guaranteed that all data remained consistent across client applications and administrative tools, while maintaining audit logs and secure access for accountability.

By using Vue.js for the Admin Portal, the development team benefited from:

- **Reactive User Interfaces:** Automatic updates to the user interface when underlying data changed, improving administrator efficiency and user experience.
- **Component-Based Architecture:** Modular and reusable components for faster development and easier maintenance.
- **Seamless Integration with Backend APIs:** Efficient handling of JSON data and API requests, enabling real-time monitoring and control of the platform.

Overall, the Admin Portal served as a central control hub for the system, complementing the client applications and backend services, while ensuring secure, efficient, and scalable management of the platform.

3.3.2 Technology Stack & Rationale

The technology stack had been carefully chosen to prioritize rapid development, cost-efficiency, reliability in Uganda's environment, and future scalability of the platform. Each component was selected based on its performance, maintainability, and suitability for mobile-first transportation services.

- **Cross-Platform Mobile Development: Flutter + Dart**
  Flutter was a modern, open-source framework developed by Google, using the Dart programming language. It allowed a single codebase to run on both Android and iOS devices, significantly reducing development time and maintenance efforts. Dart provided a strongly typed, object-oriented environment optimized for building reactive user interfaces. Flutter was particularly suitable for Uganda, where low- and mid-range devices (Tecno, Itel, Infinix) dominate the market, as it delivered high performance and smooth UI animations. Built-in Material Design components ensured a consistent and visually appealing user experience across platforms.
- **Admin Portal Development: Vue.js**
  Vue.js was a progressive JavaScript framework that enabled the development of reactive, modular, and maintainable web interfaces. The Admin Portal was built using Vue.js to provide administrators with a responsive web interface for user management, system monitoring, and analytics dashboards. Vue's component-based architecture allowed efficient development, rapid iteration, and easy integration with backend REST APIs.
- **Backend & RESTful API: Django + Django REST Framework (DRF)**
  Django, a high-level Python web framework, served as the core backend. Python was widely known for its readability, extensive library ecosystem, and suitability for rapid development. Django provided a secure and production-ready framework with a built-in admin interface, ORM for database management, and strong security practices. The Django REST Framework exposed RESTful APIs, enabling secure, scalable, and standardized communication between client applications, the admin portal, and external services.

- **Real-time Features: Firebase Realtime Database / Firestore + Push Notifications**
  Firebase supported real-time data synchronization and push notifications. This ensured sub-second location updates for rides, immediate push notifications for drivers and passengers, and consistent system state even on low-bandwidth 2G/3G networks. Firebase also offered a generous free tier for MVP deployment, minimizing costs during early stages while allowing seamless scaling as the user base grew.

- **Primary Database: PostgreSQL with PostGIS extension**
  PostgreSQL was a robust, open-source relational database chosen for its reliability, ACID (Atomicity, Consistency, Isolation, Durability) compliance, and strong support for complex queries. The PostGIS extension added advanced geospatial capabilities, enabling efficient nearest-driver calculations, route mapping, and stage boundary management. Its open-source nature reduced licensing costs while providing enterprise-grade features.

- **Mapping & Geocoding: Google Maps Platform + OpenStreetMap (OSM) fallback**
  Google Maps provided accurate mapping, routing, and geocoding for a smooth initial user experience. OpenStreetMap tiles were cached locally to provide offline functionality and reduce future operational costs, ensuring the system remained usable in low-connectivity areas common in Uganda.

- **Authentication & Phone Verification: Firebase Authentication**
  OTP-based phone number authentication was employed to simplify user login and verification. This approach was ideal for the Ugandan context, where the majority of users had mobile phone numbers but might not have email addresses. Firebase ensured secure, reliable, and scalable authentication without the need to implement custom verification systems.

- **AI Features: MCP (Model Context Protocol)**
  MCP was a Python-based AI services framework that provided advanced intelligence features such as predictive ride matching, demand forecasting, and natural language processing. By integrating MCP with the backend, the system leveraged AI to optimize ride allocations, enhance user experience, and support data- driven decision-making.

- **Compute-Intensive Services: AWS (Amazon Web Services)**
  AWS was utilized for tasks that required scalable cloud computing resources, such as voice-to-text transcription and language translation. AWS ensured reliable, high-performance execution of these tasks without burdening the primary backend servers, enabling smooth operation even under high load.

- **Offline Support & Queueing Strategy: Hive + Work Manager / Background Sync**
  Hive, a lightweight local database for Flutter, enabled offline data storage and caching. Combined with background task managers, all actions performed offline (such as ride

requests or location updates) were automatically synced with the backend once connectivity was restored. This strategy was critical for routes with intermittent network coverage.

- **Hosting & Deployment:**

  - Backend → **Contabo cloud servers**, providing cost-effective and reliable cloud hosting with flexibility for scaling.
  - Mobile applications were distributed via the Google Play Store for general users, while direct APK sideloading was supported for drivers who may not have access to the Play Store. The Apple App Store was used for users on iOS devices.
  - Admin Portal was hosted on the same backend infrastructure and was accessible through web browsers.

This technology stack ensured a balance between **rapid development**, **cost efficiency**, **reliability in low-connectivity regions**, and **future scalability**, making it well-suited for a transportation platform tailored to the Ugandan market.

3.3.3 System Architecture

The system architecture provided a high-level representation of the interactions and relationships between the main entities of the platform. It was designed to ensure scalability, maintainability, data consistency, and security while enabling efficient communication across all system components.

The architecture followed a **layered client-server model**, which divided the system into distinct layers: client applications, an administrative portal, a centralized backend, a data storage layer, and external services.

Layered Architecture Overview

1. **Client Layer:** This layer included the Passenger App and Driver App, developed using Flutter (Dart). The client layer handled all user interactions, ride requests, real-time notifications, and location updates. By separating the client from the server, the system ensured that the user interface could evolve independently from business logic, allowing faster iterations and platform-specific optimizations.

2. **Admin Layer:** The Admin Portal, developed using Vue.js, provided administrators with a responsive, web-based interface for monitoring, managing users, controlling system configurations, and accessing analytics dashboards. The separation of the admin interface from the client apps enhanced security by limiting direct access to sensitive system operations.

3. **Backend Layer:** The Django-based backend served as the **single source of truth**, managing business logic, authentication, passenger–driver matching, AI requests, notifications, and system analytics. The backend layer ensured data integrity, enforced

rules consistently, and centralized core functionality, simplifying debugging and maintenance.

4. **Data Layer:** PostgreSQL provided reliable, persistent storage for all critical data, including user accounts, trip histories, logs, and operational metadata. Centralized data storage ensured strong consistency, transactional integrity, and easier backup and recovery.

5. **External Services Layer:** Third-party services were integrated to enhance functionality:

   - **Firebase** for authentication and real-time notifications.
   - **Google Maps API** for geolocation, routing, and mapping.
   - **AWS Services** for compute-intensive tasks like transcription.
   - **MCP (Model Control Protocol)** to enable AI-driven features.

Interaction Overview

- The App communicates with the backend to send ride requests, location updates, and receive notifications in real time.
- The Admin Portal queries the backend for system data, performed administrative operations, and managed configurations. All actions are logged for accountability.
- External services are accessed through secure backend APIs or directly by clients when necessary, enabling geolocation, messaging, AI processing, and transcription features.

Architecture Justification

The layered client-server architecture had been chosen over alternatives such as monolithic, peer-to-peer, or fully microservices-based architectures due to several key advantages:

- **Separation of Concerns:** Each layer focused on a specific responsibility, reducing complexity and simplifying maintenance.
- **Scalability:** The backend and data layers could be scaled independently to handle increased traffic or additional users without affecting the client or admin layers.
- **Security:** Sensitive operations, such as data management and AI processing, were centralized in the backend, allowing better access control and reducing the attack surface.
- **Maintainability and Modularity:** Changes in one layer (e.g., updating the mobile UI or the admin portal) did not impact the others, allowing parallel development and faster deployment.
- **Integration of Third-Party Services:** The centralized backend facilitated consistent integration with services like Firebase, AWS, and Google Maps.

System Architecture Diagram

A high-level system architecture diagram illustrated:

- Passenger and Driver Apps communicating with the Django backend via RESTful APIs.
- Admin Portal interacting securely with the backend for administrative operations.
- Backend coordinating data flow between PostgreSQL, AI services (MCP), Firebase, AWS, and Google Maps.
- Real-time notifications and AI-powered features integrated seamlessly into client interactions.

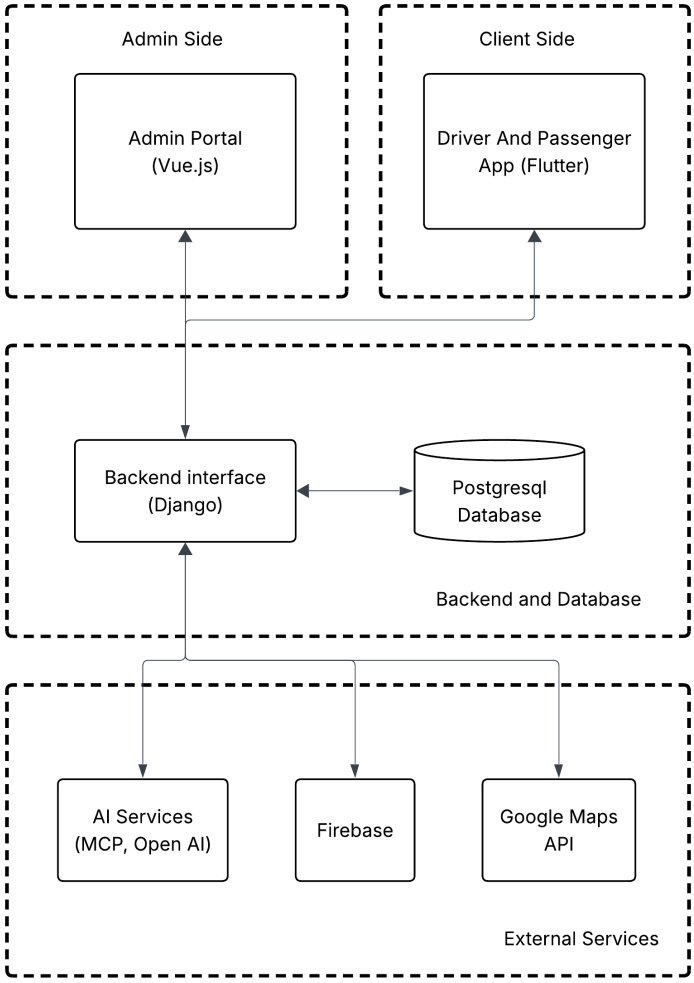


Figure 3.1: Methodology diagram of the entire system flow

3.3.4 Implementation

The Traxis system was developed using an Agile-inspired iterative approach, structured around three focused sprints of approximately 3–4 weeks each. This methodology ensured continuous feedback, early delivery of functional components, and adaptability to evolving requirements. Development of the Passenger and Driver applications was performed from a single Flutter codebase, guaranteeing 100% feature parity and identical behavior on both Android and iOS platforms from the outset.

- **Sprint 1 – Foundation (Weeks 1–4)**
  The first sprint established the core infrastructure of the system. This included:
  - Backend scaffolding with **Django** and **Django REST Framework**, creating the initial API endpoints and authentication logic.
  - Database setup with **PostgreSQL** and the **PostGIS** extension, supporting geospatial queries for future ride matching and route calculations.
  - Integration of **Firebase Authentication** for secure phone- number-based login and **Cloud Messaging** for real-time notifications.
  - Implementation of the real-time location pipeline to track passenger and driver positions efficiently.
  - Initial setup of **MCP (Model Control Protocol)** for AI service integration, preparing APIs for future predictive ride matching and AI-based recommendations.
  - Configuration of **AWS services** for compute-intensive tasks such as transcription, batch processing, and AI computations.

- **Sprint 2 – Core Interaction (Weeks 5–8)**
  The second sprint focused on user-facing functionality and real-time interactions:
  - Development of **Passenger and Driver UI components** using Flutter, including ride request flows, driver availability status, and passenger booking screens.
  - Implementation of phone-number login with OTP verification, leveraging Firebase Authentication.
  - Driver availability broadcasting and passenger request handling via the backend and Firebase Cloud Messaging.
  - Aggressive local caching using **Hive** to maintain functionality in low-connectivity or intermittent network areas.
  - Real-time push notifications for ride status updates and driver- passenger communication.
  - Integration of MCP-based AI features into core flows, including predictive driver matching and demand forecasting, communicating via backend APIs.
  - Offloading heavy processing tasks to AWS services, such as voice- to-text transcription for ride requests.

- **Sprint 3 – Polish & Advanced Features (Weeks 9–12)**
  The third sprint added advanced features, optimization, and system polish:
  - Heatmap visualization for demand prediction and operational insights, leveraging MCP AI analytics.
  - Trip history and detailed ride analytics for passengers and drivers.
  - Rating and feedback system to ensure service quality.
  - Admin portal enhancements, including dispute resolution tools and reporting dashboards.
  - Performance tuning for backend and mobile apps, including optimization of AWS-backed compute tasks and MCP API calls.
  - Accessibility features, such as voice prompts for drivers, high- contrast UI modes, and intuitive navigation.

This implementation plan ensured that core functionality was delivered early, enabling iterative testing and feedback while progressively integrating advanced AI features and cloud-based services. By following this structured Agile approach, the development team maintained high code quality, ensured system reliability, and supported rapid adaptation to emerging user and business needs.

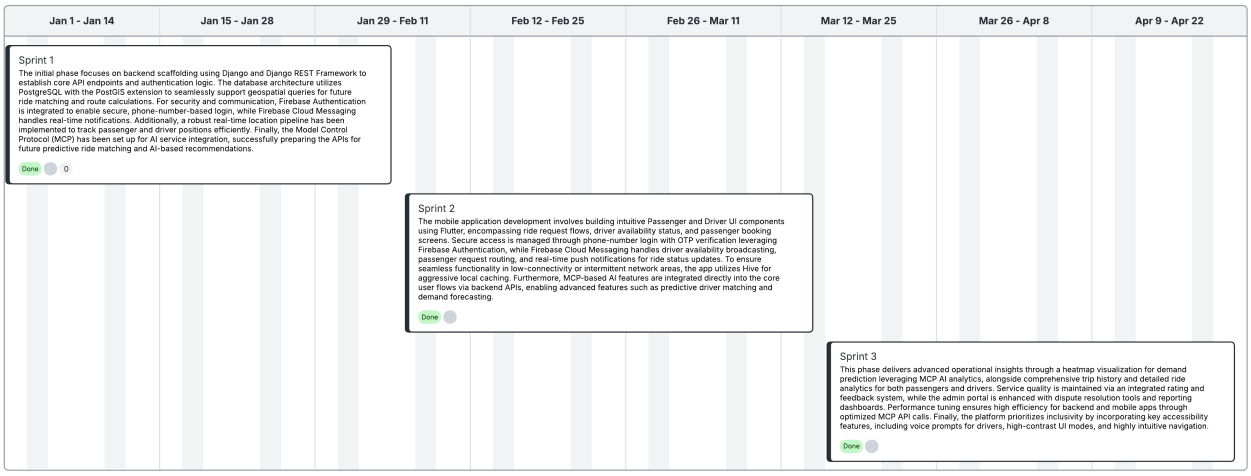


Figure 3.2: Gantt Chart showing the intended work plan

3.3.5 Project Management & Version Control

Version Control: GitHub
The project utilized GitHub for source code management and version control. A structured branching strategy was employed, including a `main` branch for production-ready code and a `develop` branch for ongoing development. All feature additions, bug fixes, and improvements were implemented through pull requests (PRs), which underwent code review before

merging. This approach ensured code quality, traceability, and collaborative development while minimizing integration errors.

Project Management: ClickUp

ClickUp was used to plan, track, and manage all project activities. The platform supported the Agile sprint structure with task assignments, deadlines, and progress tracking through burndown charts. Automated reminders and notifications helped prevent task slippage, ensuring timely completion of features and alignment with the overall project timeline. Additionally, ClickUp provided transparency and accountability for all team members, facilitating efficient communication and coordination throughout the project lifecycle.

Chapter 4

System Analysis and Design

This chapter describes the field study , analysis and design of the system. It highlights the identified requirements and the architectural design of the system.

4.1 Field Study

The field study was carried out at Uganda's Hills Link Express, a private shuttle service operating seven- and fourteen-seater vans along the Kampala–Kabale route. The primary objective was to understand how the private shuttle transport system currently operates and to identify the specific inefficiencies that a digital solution could address.

Findings revealed that the system in use is largely manual and heavily dependent on informal communication processes. To book a ride, prospective passengers are required to contact the shuttle operator via phone calls. As a result, the business owner relies on multiple phones to handle and manage booking requests simultaneously. Booking confirmation is also conducted manually; payments are typically made in cash, and in cases where customers use digital platforms such as MTN MoMo Pay, the operator takes a photo of the transaction receipt and sends it to the customer as proof of payment. This approach is inefficient, unstructured, and difficult to manage, particularly as demand increases.

Further analysis of the system revealed several key operational challenges, which are discussed below.

- **Lack of a structured booking process:** The booking process lacks structure and accountability. Customers are able to reserve seats but may fail to show up without prior cancellation or communication. This results in revenue loss and underutilised seats. Additionally, the reliance on phone-based communication places a significant burden on the operator, who must continuously respond to booking enquiries, location updates, and delivery status requests. This form of communication is not scalable and becomes increasingly difficult to manage as the customer base grows.

- **Unreliable seat allocation and poor seat utilisation:** The current system does not provide a transparent or reliable mechanism for seat allocation. Although passengers may request specific seats, such as front seats, these requests are not guaranteed, as operators may reassign seats based on personal preferences or customer relationships.

This creates a lack of trust and transparency in the booking process. In addition, seat utilisation is inefficient. Passengers may disembark at intermediate locations such as Mbarara, leaving empty seats that are not reassigned for the remainder of the journey. The absence of a system to dynamically match available seats with passengers along the route leads to missed revenue opportunities and reduced operational efficiency.

- **Limited service visibility and discoverability:** The service relies primarily on word-of-mouth referrals and radio advertisements, making it difficult for new customers to discover and access the service. This limits the business's reach and growth potential. Additionally, there is no centralised platform where customers can view available routes, schedules, or pricing information. Accessibility is further affected by varying levels of digital literacy among users. While some customers may prefer digital solutions, others rely on phone calls, highlighting the need for a system that accommodates both interaction methods.
- **Absence of parcel tracking:** The shuttle service also handles parcel delivery; however, there is no system for tracking shipments in real time. Customers frequently contact the operator to enquire about the status and location of their goods, increasing the communication burden. In addition, there is no structured approach to managing the balance between passenger transport and cargo. Vehicles may carry either passengers or goods depending on demand, but without a system to optimise this allocation, potential efficiency gains are lost.
- **Limited operational visibility for the business owner:** The current system provides limited visibility for the business owner. It is difficult to track vehicle movement, monitor driver activity, verify reported revenue, or determine seat occupancy in real time. This lack of operational insight reduces the owner's ability to make informed decisions and effectively manage the business. Return trips present an additional challenge, as demand is often low. Without visibility into passenger demand, drivers may remain at the destination rather than returning, leading to inefficiencies in service utilisation.
- **Lack of passenger accountability and safety records:** The system lacks structured mechanisms for tracking passengers and ensuring accountability. There is no reliable record of who was on a vehicle, where passengers boarded or disembarked, or emergency contact information. This creates potential safety risks and limits the ability to respond effectively in case of incidents.

4.2 System Analysis

4.2.1 Data Analysis and Interpretation of Results

The findings from the system study reveal that the current private shuttle transport system is largely informal, unstructured, and inefficient in handling core operational processes. The heavy reliance on manual and phone-based interactions limits the scalability of the service and increases the likelihood of errors, miscommunication, and operational delays.

A key issue identified is the lack of a structured booking system, which results in unreliable seat reservations and frequent no-shows. This directly affects revenue generation and reduces overall efficiency. In addition, the absence of real-time seat management prevents the system from reallocating seats when passengers disembark mid-journey, leading to underutilisation of available capacity.

Communication within the system is also inefficient, as the operator is required to handle all customer interactions manually. This creates bottlenecks and limits the ability of the business to expand. Similarly, the lack of service visibility restricts customer access, as potential users have no easy way to discover routes, schedules, or availability.

From a logistics perspective, the absence of a tracking system for both passengers and parcels results in constant enquiries and increased workload for the operator. Furthermore, the inability to balance passenger and cargo transport reduces operational optimisation opportunities.

The analysis also highlights limited operational control and visibility for the business owner. Without real-time monitoring of vehicles, revenue, and occupancy, it is difficult to make informed decisions or verify driver activity. Safety concerns are also evident due to the lack of structured passenger records and traceability mechanisms.

Overall, the current system demonstrates significant gaps in coordination, transparency, and efficiency. These findings justify the need for a digital solution that can automate booking, improve communication, enable real-time tracking, and provide better operational control.

4.2.2 User Requirements

The following are the requirements the users expect Traxis to meet:

1. Users need a simple way to book shuttle seats without making phone calls.
2. Passengers need the ability to select specific seats.
3. Users require confirmation after making a booking or payment.
4. Passengers need to know shuttle location and arrival times.
5. Customers need a way to track parcels during delivery.
6. Operators need a way to monitor vehicle movement and seat occupancy.
7. The system should reduce excessive phone communication.
8. The system should be easy to use for individuals with low digital literacy.

4.2.3 Functional Requirements

The system was required to provide the following core functionalities:

1. Allow passengers to create accounts using phone numbers.

2. Enable users to search for available shuttle routes and schedules.
3. Allow passengers to book specific seats on a shuttle.
4. Provide real-time updates on seat availability.
5. Enable passengers to make payments and receive automated confirmation.
6. Allow users to cancel bookings based on defined conditions.
7. Provide real-time tracking of shuttle location.
8. Enable passengers to receive notifications on trip status and updates.
9. Allow operators to manage bookings and update seat availability.
10. Enable operators to monitor vehicle movement in real time.
11. Provide a dashboard for viewing revenue, trips, and occupancy levels.
12. Allow operators to update passenger and trip records.
13. Support courier service functionality, including:
    - Recording parcel details
    - Tracking delivery status
    - Notifying users upon delivery
14. Provide a hybrid communication system that supports both in-app interaction and phone-based contact.

4.2.4 Non-Functional Requirements

The system was designed to be user-friendly and accessible to individuals with varying levels of digital literacy. It should:

1. Allow users to access system features immediately after successful authentication.
2. Provide a simple and intuitive interface for easy navigation between different sections of the application.
3. Minimise user input errors through validation mechanisms.
4. Allow users to easily correct input errors before submission.
5. Provide clear feedback and notifications upon successful completion of key actions such as registration, booking, and payment.
6. Assure fluid UI animations (e.g. consistent 400ms page transitions) to maintain a premium end-user experience.

7. Efficiently handle peak traffic times, such as holiday seasons, without failing to process transactions or generate digital tickets.

8. Transmit personal user data, location details, and payment information securely using modern encryption standards.

4.2.5 System Requirements

- **Platform Compatibility:** The application must be compatible with Android 13 (API level 33) and above, as well as iOS 17 and above.
- **Network Connectivity:** An active internet connection (3G/4G/5G or Wi-Fi) is required for real-time synchronisation of location data, bookings, and notifications.
- **Hardware:** A standard mobile device equipped with location service capabilities (GPS) and a minimum of 2 GB RAM.

4.3 System Design

This section includes a detailed description of the system's architecture, components, modules, interfaces, and data structures designed to satisfy the specified requirements. It describes the design and development process of the application using a use case diagram to explain how actors interact with the system and data flow diagrams to show how data moves in and out of the system.

4.3.1 System Architecture

TRAXIS employs a **Client-Server Architecture** built on a layered, modular design pattern. The Flutter application serves as the client interface, responsible for UI rendering, state management, and local caching, while a centralised cloud-hosted backend acts as the single source of truth for all data and business logic.

The architecture is divided into four principal layers:

1. **Presentation Layer:** Encompasses all interactive Flutter screens (e.g., `dashboard_screen`, `operator_dashboard_screen`, `digital_ticket_screen`) and UI widgets using dynamic layouts and global theming defined in `app_theme.dart`.

2. **Core / Domain Layer:** Houses the business logic, utilising providers (e.g., `auth_provider.dart`) and state management mechanisms to bridge the data and presentation layers.

3. **Data Layer:** Manages API integrations, external service calls, and secure local storage mechanisms, ensuring clean separation between business logic and data access.

4. **External Services Layer:** Integrates third-party services including Firebase (authentication, messaging, real-time sync), Google Maps API (geolocation and routing), AWS Services (compute-intensive tasks), and AI services via MCP.

Use Case Diagram

The use case diagram below illustrates the different types of users of the system and how they interact with it. Two primary actors are identified: the

Passenger , **Administrator** and the **Operator**. The diagram provides a simplified graphical representation of the system's functionalities and the interactions each actor may perform.

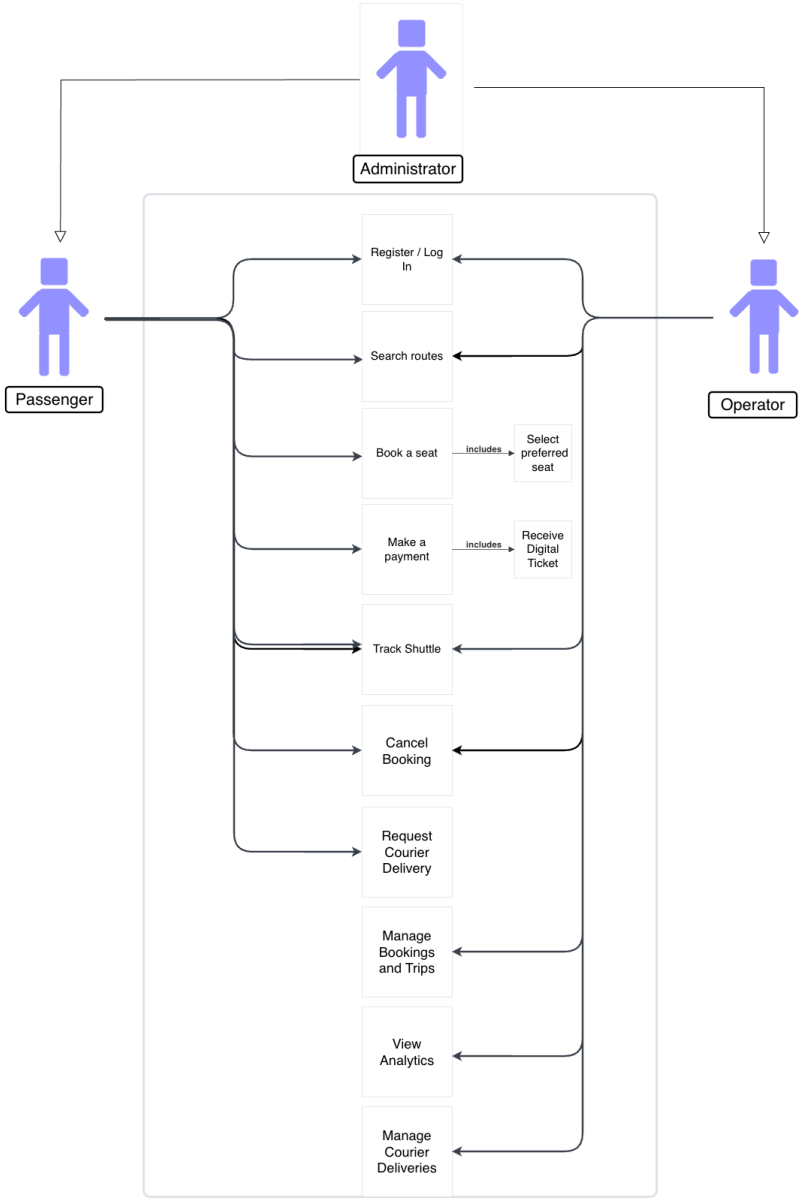


Figure 4.1: Use case diagram showing how users interact with the Traxis system

Context Diagrams

A context diagram describes the logical flow of data within the system, illustrating how each user type interacts with the system from authentication through to completing their

primary tasks. The Level 0 diagram below presents the system as a single process, showing all external entities and their data exchanges with the system.

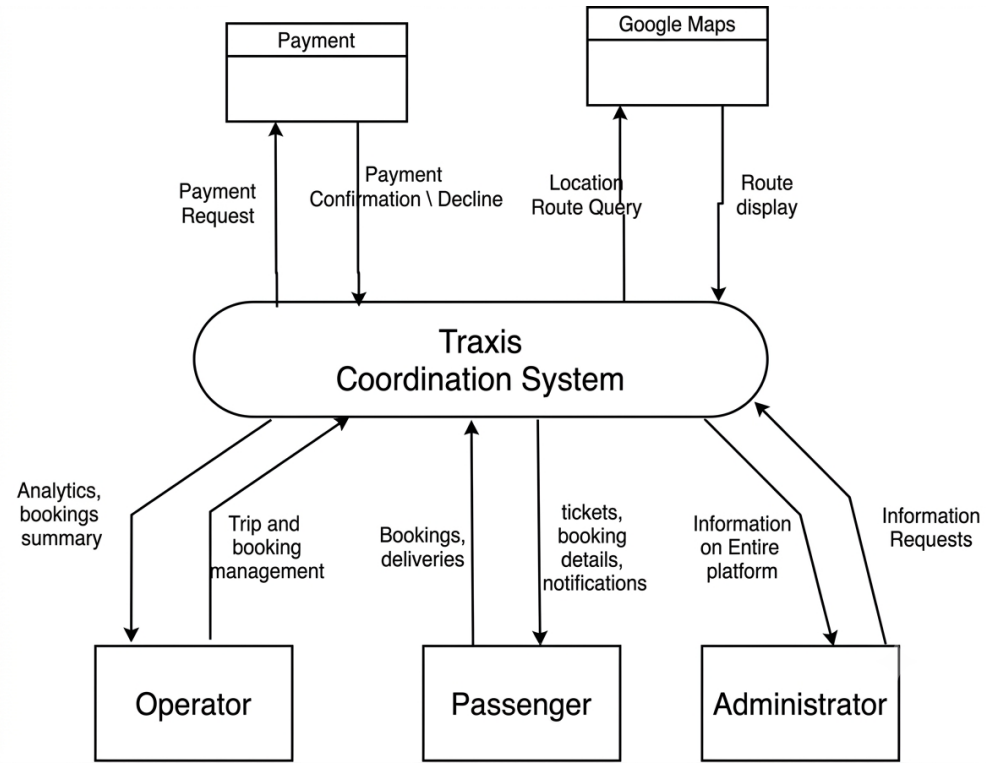


Figure 4.2: Level 0 Context Diagram showing the overall system boundary and external entities

4.3.2 Design Constraints

- **Cross-Platform Consistency:** Maintaining a uniform graphical experience across both iOS and Android natively while avoiding UI inconsistencies between the two platforms.
- **Network Dependency:** Real-time tracking and live ticketing mandate continuous internet and GPS connectivity. The system must gracefully handle offline or low-connectivity degradation, using local caching via Hive to queue actions until connectivity is restored.
- **Mobile Hardware Limitations:** Rendering complex UI animations (ambient blur orbs, gradients) and active map overlays requires meticulous memory optimisation to prevent frame rate drops or device overheating, particularly on low- and mid-range devices common in Uganda.
- **Internet Connectivity Requirement:** All location server platforms implemented require active internet connectivity for real-time synchronisation. The response time for location data is therefore dependent on the quality of the mobile network available to the user.

4.3.3 Design Methodology

We used the **Agile development methodology**. Agile Software Development (ASD) is a creative process that anticipates the need for flexibility and applies a high level of pragmatism into the delivery of the finished product. We focused on keeping code simple, testing often, and delivering functional components of the application as soon as they were ready. This helped us to minimise risks such as bugs, cost overruns, and changing requirements. Rapid prototyping of the graphical user interface was conducted first to gather immediate feedback on custom aesthetic elements. Following approval of the UI design, functional milestones were iteratively coded and deployed, linking the distinct passenger and operator workflows in sequential project sprints.

4.3.4 High Level Design

The high-level design structures the application into self-contained major subsystems:

1. **Identity and Access Management:** OTP issuance, user profile creation, role retrieval, and session token management.
2. **Passenger Operations Subsystem:** A pipeline covering: Search → Seat Selection → Payment → Digital Ticketing.
3. **Logistics and Tracking Subsystem:** Real-time vehicle geolocation and package dispatch tracking using mapping services.
4. **Operator Management Subsystem:** Dedicated to dashboard rendering, booking aggregation, and financial analytics.
5. **System Administration Subsystem:** A system administrator Dashboard. Responsible for system configuration, security, data management, maintenance, support, and ensuring system performance as

Logical Design

This view shows the logical functional elements of the system, clearly indicating the major processes involved and how they relate to one another.

At a logical level, the system separates actions driven by user roles that interact with shared data entities:

- A *Customer/Client* interacts with the system to create multiple Bookings and package requests.
- A *Operator* The operator interacts with the system to create and manage Trips, which are then booked by Customers. The Operator also manages package deliveries and monitors vehicle locations.
- The system processes these interactions to update the state of Bookings, Trips, and

- A *System Administrator* is responsible for system configuration, security,data management, maintenance,support, and ensure system performance as required by both the customer and the operator.

Process (Module) Design

This view is for project management and code organisation. The components are organised as independent modules corresponding to files or directories within the codebase. The primary functional modules implemented within the TRAXIS codebase include:

- **Authentication Module:** Manages the entry threshold securely through Role Selection and OTP Verification screens.
- **Customer Module:** Features route planning, interactive seat mapping, and generation of the boarding pass.
- **Operator Module:** Fetches enterprise-level metrics to render bookings and financial revenue on specialised dashboards.
- **Tracking and Delivery Module:** Interfaces with map SDKs to parse and display real-time coordinate updates for vehicles.

Security

Traxis enforces security at multiple levels. Authentication is handled exclusively through OTP-based phone number verification via Firebase Authentication, eliminating password-related vulnerabilities. All external communication operates strictly over HTTPS protocols. Sensitive user data, location details, and payment information are transmitted using modern encryption standards. Role-based access control ensures that passengers and operators can only access functionality appropriate to their role, with role retrieval and session token management handled by the Core/Domain layer through
`auth_provider.dart`.

4.3.5 Database Design

Operating via scalable, cloud-based data structures, the system uses two complementary storage technologies: Cloud Firestore and PostgreSQL. Cloud Firestore is a cloud-hosted NoSQL document database provided by Firebase. It stores data in collections of documents and is optimised for mobile apps, real-time synchronization, and offline-friendly updates. This makes it well suited for live state information such as vehicle locations, seat availability, and booking status changes.

PostgreSQL is an open-source relational database engine that stores structured data in tables with defined columns and relationships. It provides reliable transactions, strong consistency, and support for complex queries, so it is the primary store for durable records such as users, trips, bookings, and parcel history.

The core relations are organised as follows:

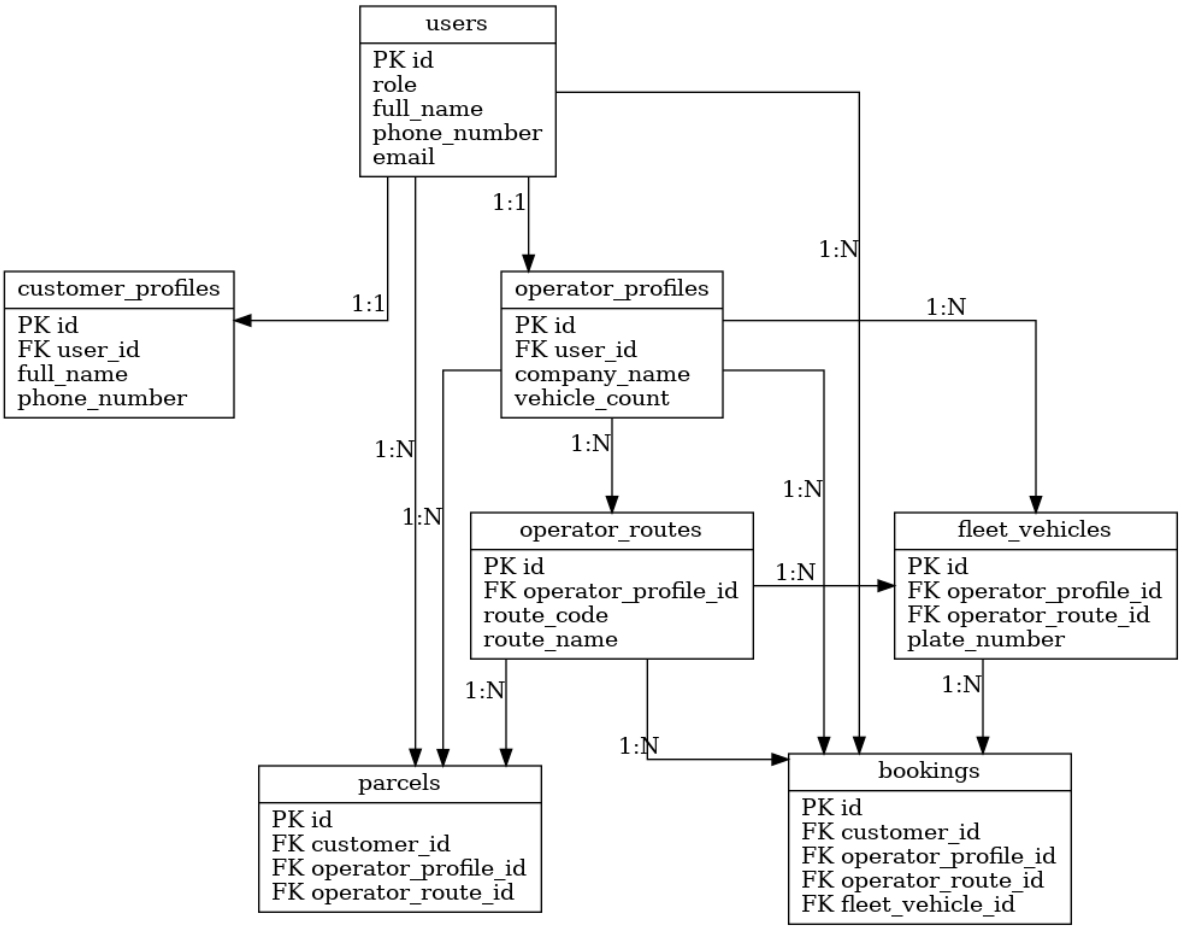


Figure 4.3: Entity-relationship diagram showing database tables and their relationships

Database Tables

- **users:** Stores all registered users and their authentication state. Key fields include `id` (primary key), `role` (one of "operator", "customer", or "admin"), `full_name`, `phone_number`, `email`, `is_active`, `is_verified`, and OTP-related fields for secure authentication. Note that the `admin` role is a flag assigned to selected users within this table and does not require a separate table.

- **customer_profiles:** Stores additional onboarding details for users with the "customer" role. Maintains a one-to-one relationship with `users` via `user_id` (foreign key), including fields such as `full_name` and `phone_number`.

- **operator_profiles:** Stores company and operations details for users with the "operator" role. Also maintains a one-to-one relationship with `users` via `user_id`. Includes fields such as `company_name`, `company_registration_number`, `company_address`, `vehicle_count`, `trip_routes`, `route_charges`, and a boolean `delivers_parcels` flag.

- **operator_routes:** Defines operational routes established by an operator. Each route belongs to one operator profile (many-to-one relationship via `operator_profile_id`).

Includes fields such as `route_code`, `route_name`, `origin`, `destination`, `via`, `base_fare`, and `estimated_duration_minutes`.

- **fleet_vehicles:** An inventory of physical vehicles owned by operators, used to determine booking seat capacity. Each vehicle is assigned to an operator profile (many-to-one via `operator_profile_id`) and may be assigned a default route (optional many-to-one via `operator_route_id`). Fields include `vehicle_code`, `vehicle_type`, `plate_number`, `seat_capacity`, and `is_active`.
- **bookings:** Stores seat bookings made by customers. Each booking references a customer (via `customer_id`), an operator (via `operator_profile_id`), a route (via `operator_route_id`), and a fleet vehicle (via `fleet_vehicle_id`). Includes fields such as `route_name`, `pickup_point`, `dropoff_point`, `travel_date`, `seat_count`, `quoted_amount`, `payment_method`, `status`, and optional `notes`.
- **parcels:** Stores details of parcel deliveries requested by clients. Each parcel references a customer, an operator, and a route (via foreign keys). Includes fields such as `tracking_number`, `origin`, `destination`, `recipient_name`, `recipient_phone`, `description`, `weight_kg`, `declared_value`, `delivery_speed`, `quoted_amount`, and `status`. Status transitions track the lifecycle: booked → picked_up → in_transit → delivered (or cancelled at any stage), with corresponding timestamp fields for each transition.

Adding Data

Data is written to the backend through RESTful API endpoints exposed by the Django REST Framework. A RESTful API is a standard web interface where the client sends JSON requests to named server endpoints and receives JSON responses. This design keeps the mobile application independent from the exact server implementation.

For real-time features, the same backend writes are simultaneously synchronised to Cloud Firestore using the Firebase SDK. Each record is identified by a unique ID generated at the point of creation. For example, when a passenger confirms a booking, a `booking_id` is generated, the corresponding seat record is updated to reflect occupancy, and a digital ticket entry is created — all within a single transactional API call to ensure data consistency.

Reading Data

Data is retrieved from the backend via RESTful API calls authenticated using session tokens. For real-time features such as live vehicle tracking and seat availability updates, the application subscribes to Firestore document streams. Changes to tracked documents (e.g. a vehicle's GPS coordinates or a seat's occupancy status) are pushed to all connected clients sub-second, ensuring that every passenger and operator always sees the most current system state.

4.3.6 Graphical User Interface Design

Inputs

- **Phone number and OTP:** Required on registration and login. Without valid credentials, a user cannot authenticate or access any system functionality.
- **GPS coordinates:** The application requests location permission from the device. This allows the user's position and the vehicle's position to be determined and displayed in real time.
- **Seat selection taps:** Passengers interact with an interactive seat grid to choose their preferred seat. The grid reflects real- time availability, with occupied seats visually distinguished from available ones.
- **Booking and payment details:** Users input trip preferences and payment information through validated form fields, with inline error messages displayed immediately upon detection of invalid input.
- **Package details:** For courier requests, users provide sender and receiver information, which is stored against the assigned trip.

Outputs

- **Interactive map view:** Immediately upon authentication, both passengers and operators are presented with a live map view (powered by Google Maps SDK) showing real-time vehicle positions.
- **Digital ticket:** Upon successful payment, the system generates and displays a digital boarding pass (`digital_ticket_screen`) containing booking reference, seat number, route, and departure time.
- **Status notifications:** Push notifications via Firebase Cloud Messaging alert passengers to trip status changes, payment confirmations, and parcel delivery updates.
- **Operator dashboard:** A 2×2 KPI grid displays live metrics including active bookings, revenue generated, seat occupancy rate, and active trips.
- **admin dashboard:** A comprehensive interface for system administrators that allows them to monitor system health, manage user accounts, and oversee operational metrics.
- **Tickets, Ride history and booking lists:** Scrollable list views (`ride_history_screen`, `operator_bookings_screen`) and ticket screens (`ticket_screen`)) display to present historical and current booking records with clear visual and tactile feedback for all system states to the designated users.

4.3.7 External Interfaces

Hardware Interfaces

- **Geospatial Sensors:** The application calls the device's native GPS unit to continuously fetch and render user and transport locations. The `ACCESS_FINE_LOCATION` permission is declared in the application manifest, and users must grant this permission for full tracking functionality.
- **Haptic Interfaces:** Device vibration motors are engaged on key user interactions such as seat selections or payment verifications, providing tactile confirmation of actions.

Software Interfaces

- **Device OS Integrations:** Deep interaction with the Android SDK and iOS Foundation frameworks is required to retrieve hardware permissions for location services and push notifications.
- **Mapping APIs:** Integration with the Google Maps SDK provides route polyline rendering and live vehicle tracking overlays. OpenStreetMap tiles are cached locally as a fallback for low- connectivity environments.

Communication Interfaces

- **RESTful JSON APIs:** All client-to-server communication operates strictly over HTTPS, with JSON as the data exchange format. API endpoints are managed by the Django REST Framework.
- **WebSockets / Firestore Streams:** Low-latency, bi- directional data streams via Firestore maintain the persistent real-time functionality required by the live location tracking and seat availability modules, ensuring sub-second updates even on 3G networks.
- **Firebase Cloud Messaging (FCM):** Push notifications for ride status updates, booking confirmations, and parcel delivery alerts are delivered to both passenger and operator devices through FCM without requiring custom notification infrastructure.

User Interfaces

The user-facing platform consists of dynamic, touch-responsive screens built with the Flutter framework. Core aesthetic elements include custom modern typography, a tailored colour palette featuring vibrant interactive gradients, and specialised glassmorphic overlays (blur orbs, translucent card panels). Navigation is provided through a carved-out bottom navigation bar with clear active/inactive visual states. Transition animations between application states are set to an optimal 400 ms duration, prioritising fluidity over abrupt page loads. Integral interactive elements include OTP `TextFields`, responsive map layouts, and dynamic

seating grids. The interface adopts **distinct dashboard layouts** for Operators (emphasising KPIs and operational data) and Passengers (emphasising action buttons, interactive maps, and ticketing) while rigorously adhering to a globally unified theme defined in `app_theme.dart`.

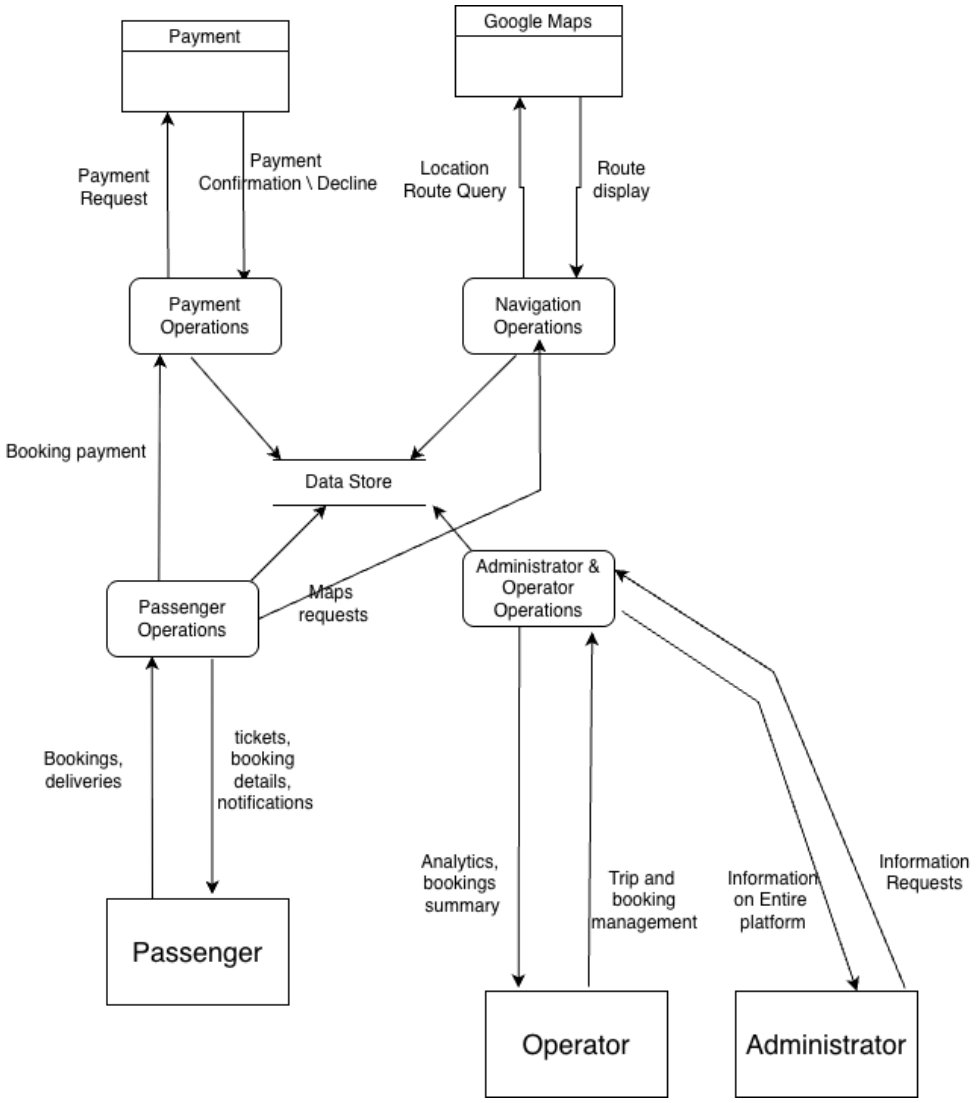


Figure 4.4: The Level 1 diagram decomposes the system further, revealing the core internal processes, data stores, and information flows between passengers, operators, and the Traxis platform.

Chapter 5

System Implementation, Testing and Validation

5.1 Introduction

This chapter describes how Traxis was realized. Traxis's implementation, testing and validation was to make sure that it meets its set objectives and the requirement specifications that were identified.

5.2 System Implementation

This section gives an overview of the project implementation. It highlights the major components and the operation of the system as well as the different user interfaces and activities that allow users to interact with the system.

5.2.1 Implementation tools

The Traxis system was implemented using a modern, scalable technology stack selected to ensure performance, reliability, and adaptability within the Ugandan transport context.

Flutter (Dart) The mobile application was developed using Flutter, a cross-platform UI framework by Google, alongside the Dart programming language. Flutter enabled the development of a single codebase that runs on both Android and iOS devices, reducing development time and maintenance effort. Its widget-based architecture supported the creation of responsive and user-friendly interfaces suitable for a wide range of devices, including low and mid-range smartphones commonly used in Uganda.

Django and Django REST Framework (DRF) The backend system was implemented using Django, a high-level Python web framework, together with the Django REST Framework for API development. Django handled business logic, request processing, and system coordination, while DRF exposed RESTful APIs that facilitated secure communication between the mobile applications, admin portal, and external services.

PostgreSQL (with PostGIS) PostgreSQL was used as the primary relational database for persistent data storage. It provided strong data integrity, reliability, and support for

complex queries. The PostGIS extension enabled geospatial data processing, which was essential for location-based features such as route mapping, nearest-vehicle identification, and geographic queries.

Firebase Services Firebase was integrated to support real-time features and user management:

Firebase Authentication was used for secure user registration and login, particularly through phone-number-based authentication, which aligns with local user accessibility. Firebase Cloud Messaging (FCM) enabled real-time push notifications, such as booking confirmations, trip updates, and alerts for both passengers and drivers.

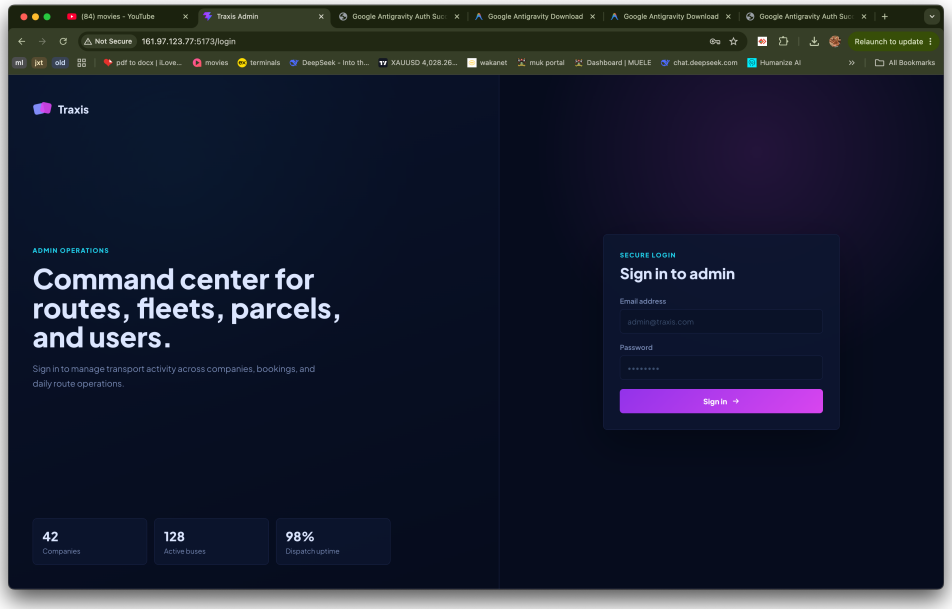
Google Maps API The Google Maps Platform was used to provide mapping, navigation, and geolocation services. It enabled real-time vehicle tracking, route visualization, and distance calculations, improving user interaction and operational coordination within the system.

AWS (Amazon Web Services) AWS was utilized for compute-intensive operations such as transcription and other backend processing tasks. Its scalable cloud infrastructure ensured efficient handling of resource-demanding processes without affecting overall system performance.

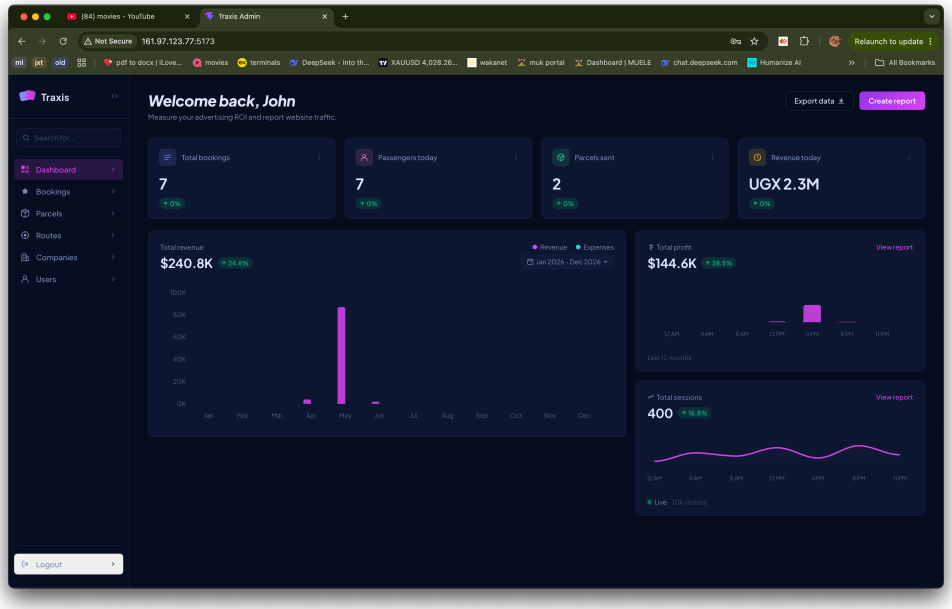
MCP (Model Control Protocol) MCP was integrated to support AI-driven functionalities, including predictive analytics and intelligent system behavior. Implemented using Python, it enabled advanced features such as demand forecasting and improved coordination between passengers and shuttle operators.

Hive and Background Sync Mechanisms Hive, a lightweight local database for Flutter, was used to support offline functionality. Combined with background synchronization tools, it allowed the system to queue actions performed offline and automatically sync them once network connectivity was restored—an essential feature for areas with unstable internet access.

Vue.js (Admin Portal) The Admin Portal was developed using Vue.js, a progressive JavaScript framework. It provided administrators with a responsive interface for managing users, monitoring trips, viewing analytics, and configuring system settings. Its reactive architecture enabled real-time updates and efficient interaction with backend APIs.

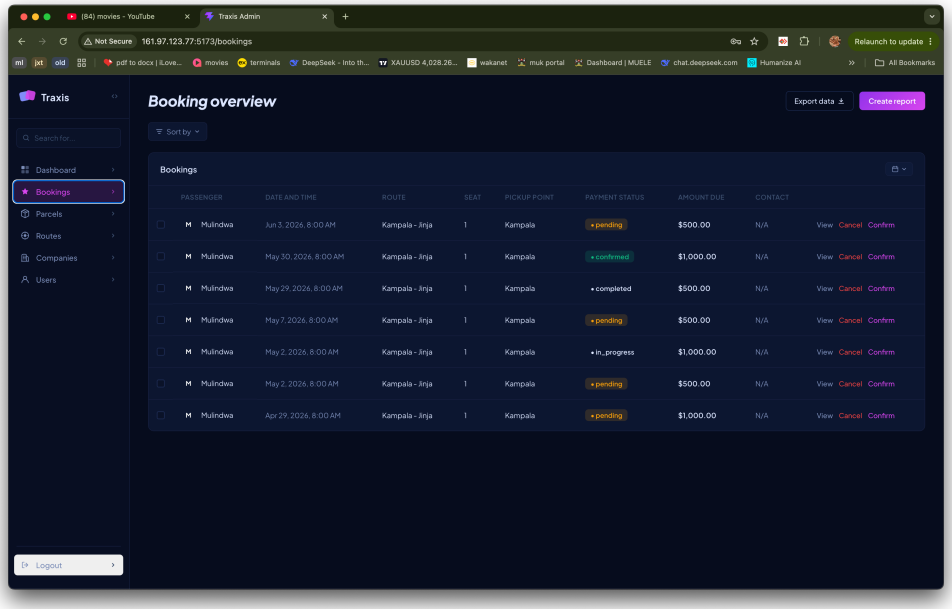


(a) Administrator dashboard Login Screen

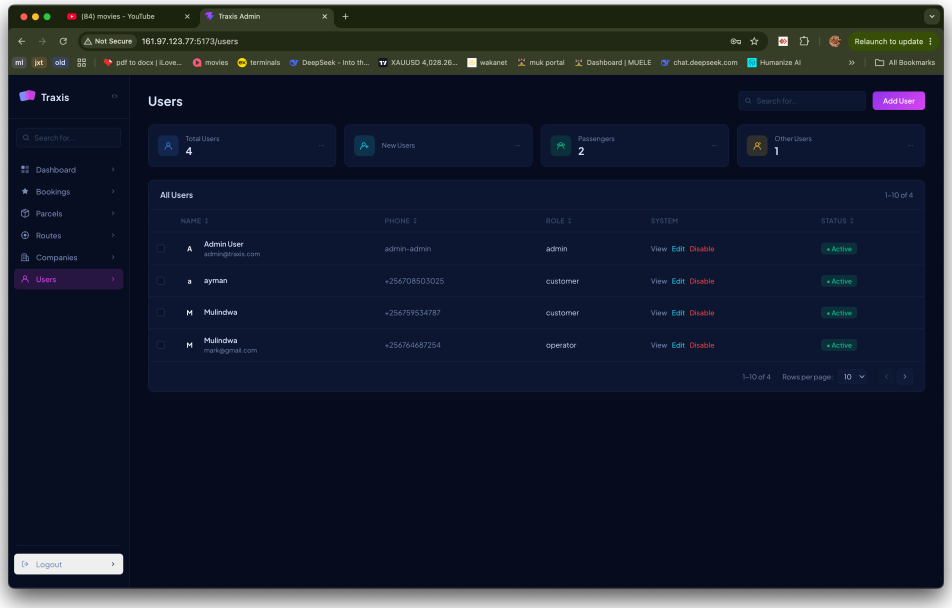


(b) Administrator Overview Screen

Figure 5.1: Administrator interfaces



(c) Administrator Bookings Overview Screen



(d) Users Screen

Figure 5.1: Administrator interfaces (continued)

5.2.2 User interface design

The system can be launched from the phone just like any other Android or IOS applications and after it has been launched, a login interface appears. This is helpful in authenticating users

Account Creation

Passengers can create an account by providing their phone number and a next of kin's phone number, which is verified through a one-time password (OTP) sent via SMS. Operators are asked to provide an E-mail address in addition to their phone number. This method ensures a secure and straightforward registration process, particularly suited for users in Uganda who may not have access to email-based authentication.

Operator Actions

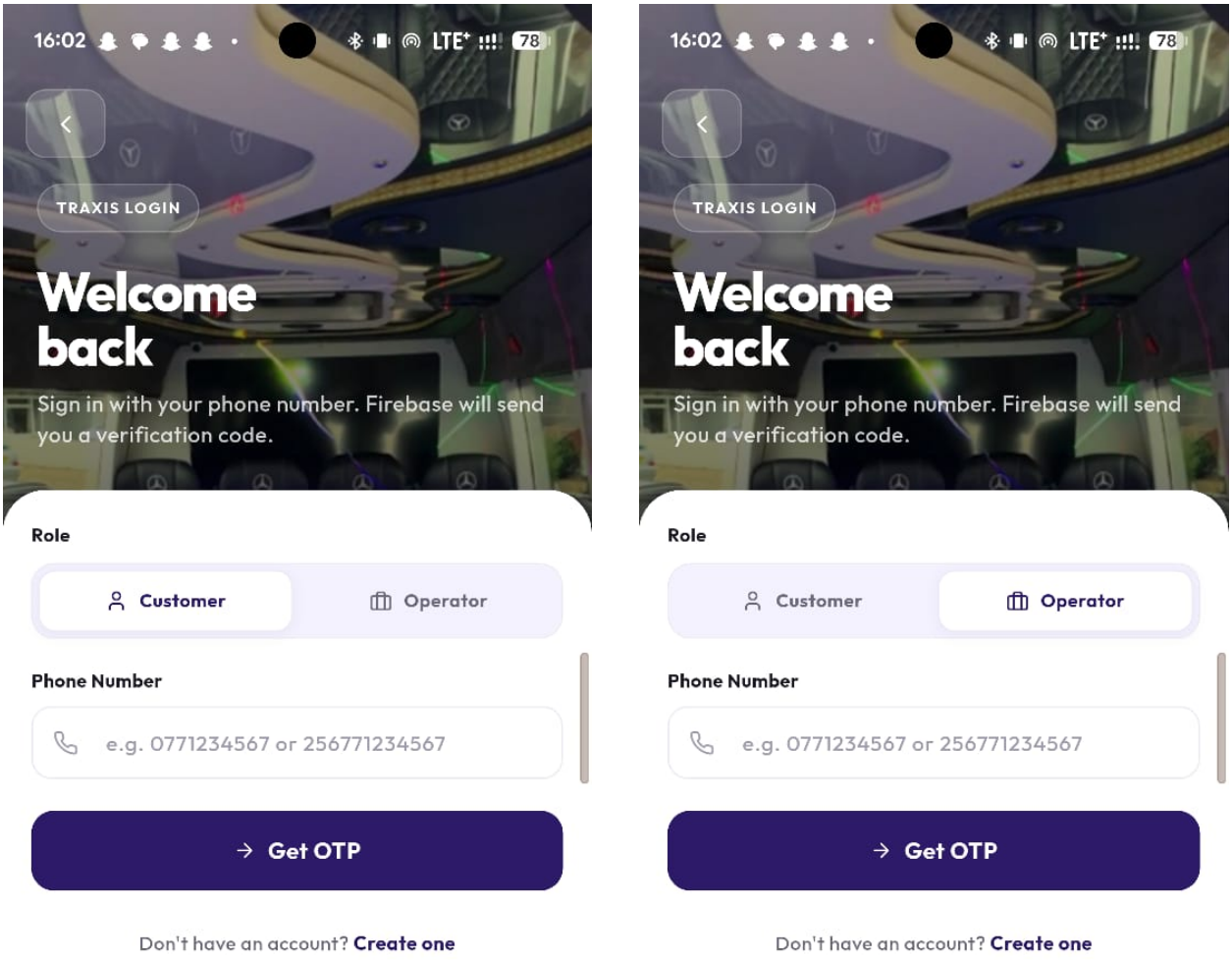
Once logged in, operators can access the main dashboard, which provides options for booking rides, viewing trip history, and managing their profiles. The interface is designed to be intuitive and user-friendly, with clear navigation and accessible features. Operators can register a company, the number of vehicles they have,the routes they operate on, and how much they charge for specific routes. They can also view and manage their bookings and indicate whether or not they support parcel delivery. On the dashboard, they can also view the number of passengers that have booked rides with them and the total revenue earned. They can add new vehicles to their fleet and view the details of their vehicles such as their location during trips and the number of trips they have made. Operators can also view the details of their passengers such as their name, phone number, and next of kin's phone number. They can also view the details of their trips such as the route taken, the distance covered, and the fare charged.

Passenger Actions

Passengers can book rides by their pickup and drop-off locations, choosing a route, choose a dispatcher and confirming their booking. Passengers can also choose to book a ride for a future date and time.The app allows them to choose a seat in the taxi. The app provides a ticketing system that generates a unique ticket for each booking, which passengers can use to verify their ride and access the shuttle.During the trip, they can track the location of their shuttle in real-time and view the estimated time of arrival. They can also view their trip history, upcoming trips, and manage their profiles. The interface is designed to be simple and accessible, ensuring that users of varying technological proficiency can easily navigate and utilize the app's features. For parcel delivery, passengers can specify the pickup and drop-off locations, the type of parcel, the item's weight and even call for clarity on the fare to be charged. They can also track the status of their parcel in real-time and receive notifications about the delivery progress.

5.2.3 System Testing and Validation

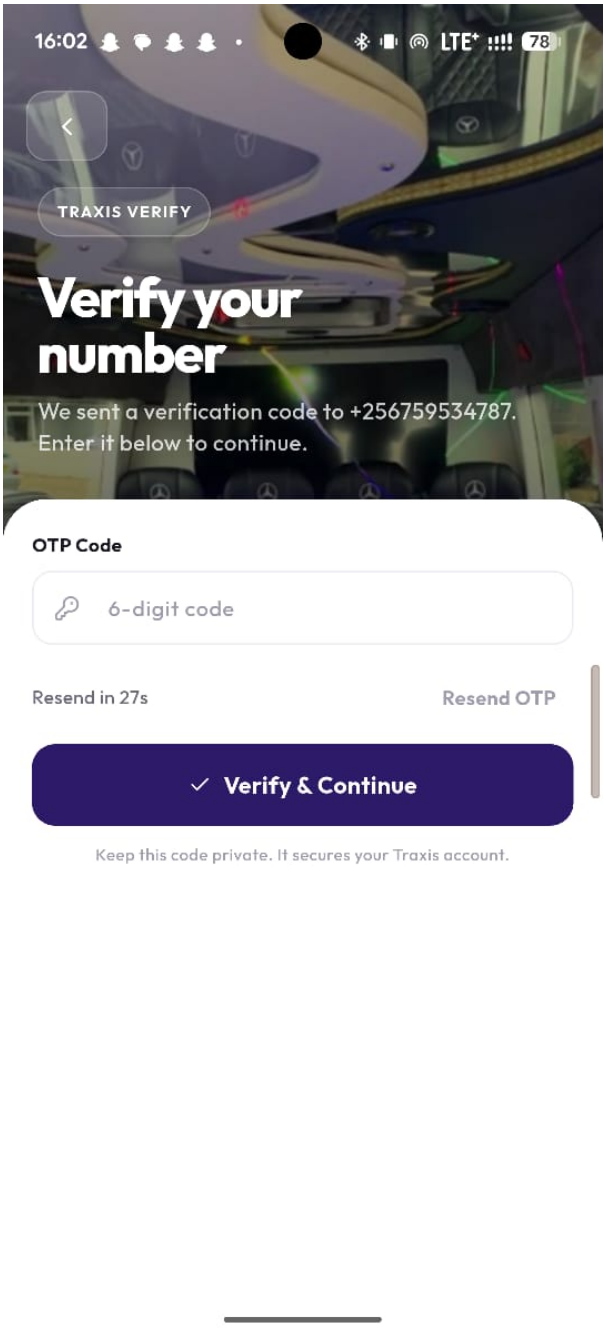
To ensure a robust and reliable system, testing was performed continuously at every stage of development. The approach emphasizes early detection of defects and validation of functionality rather than deferring testing until the end of the project. This proactive strategy guarantees that issues are identified and resolved promptly, maintaining high system quality and reliability.



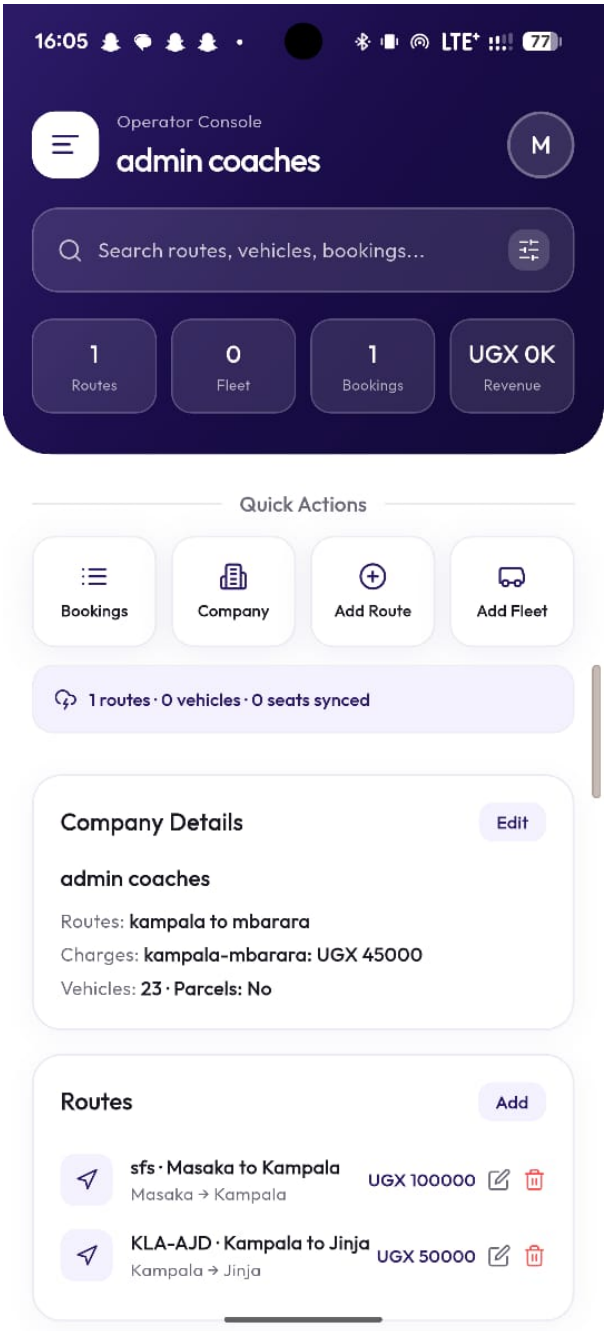
(a) Login screen

(b) Operator login screen

Figure 5.2: Authentication interfaces

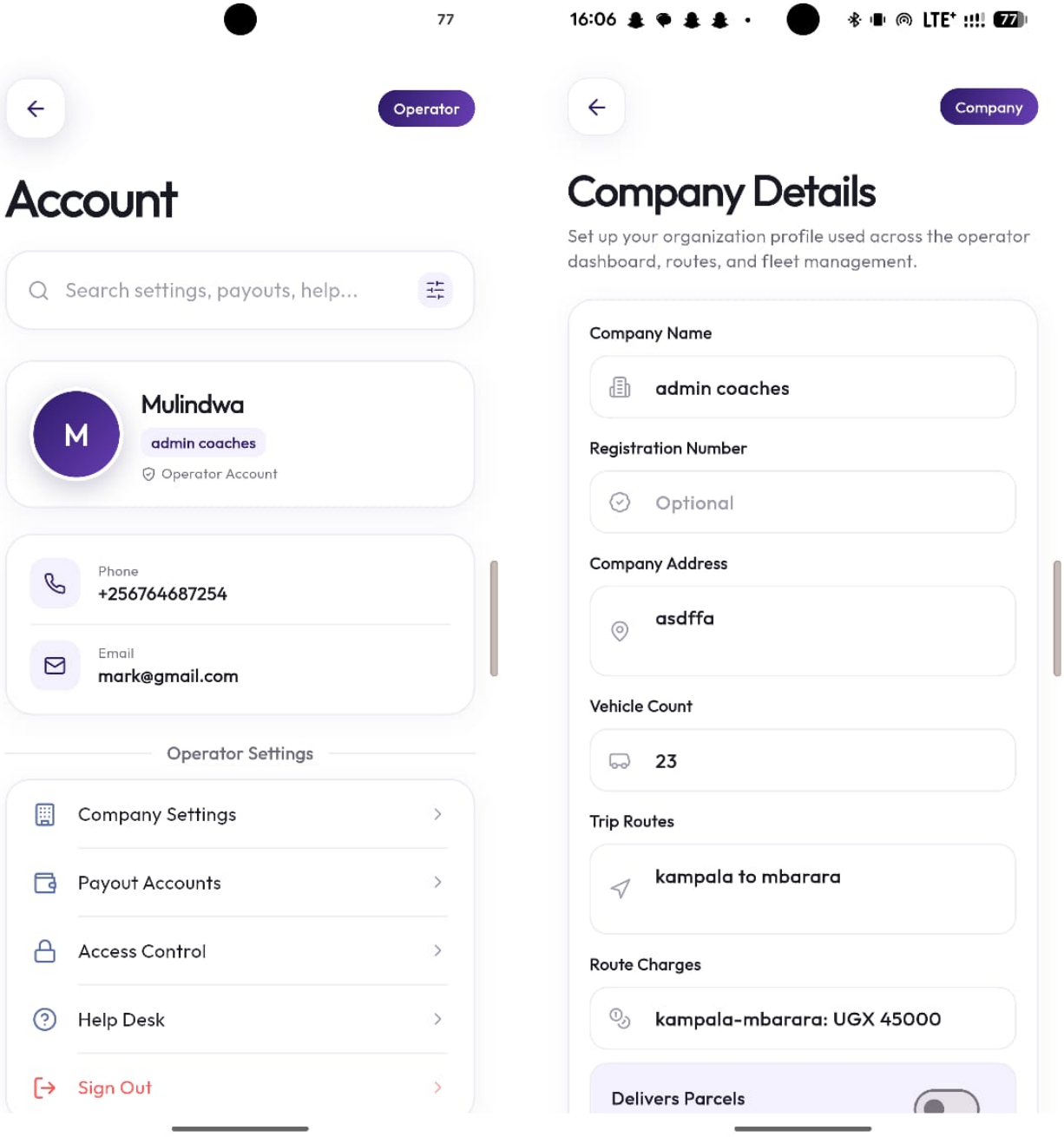


(a) OTP verification



(b) Operator dashboard

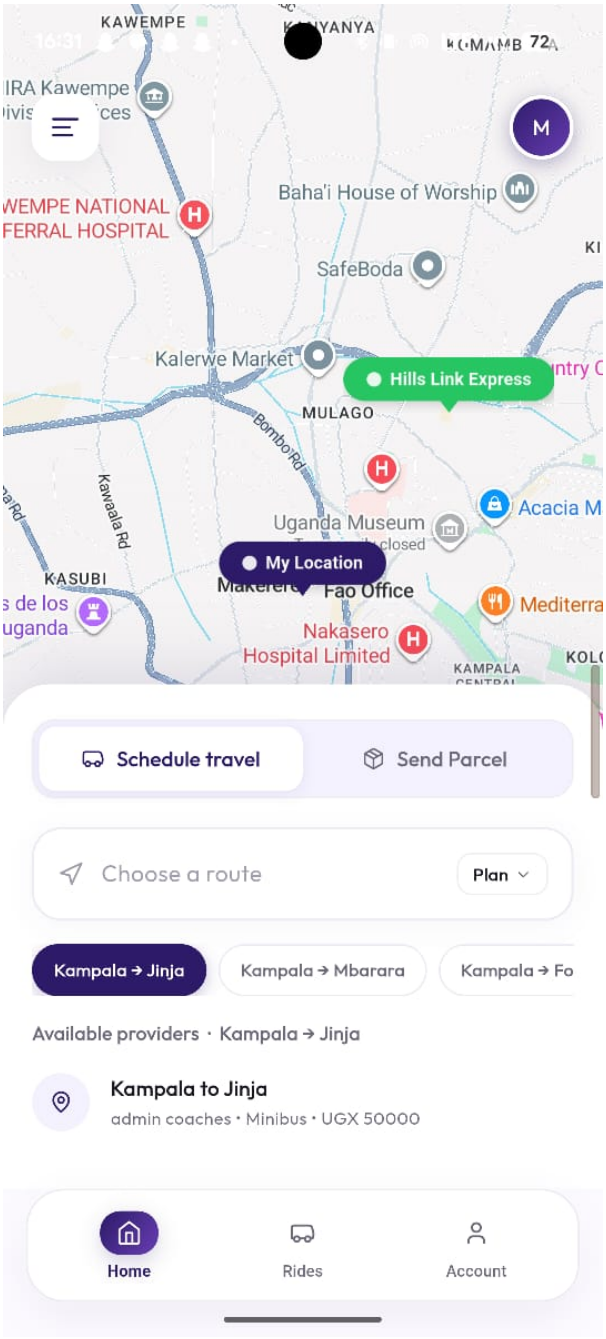
Figure 5.3: Operator login and dashboard



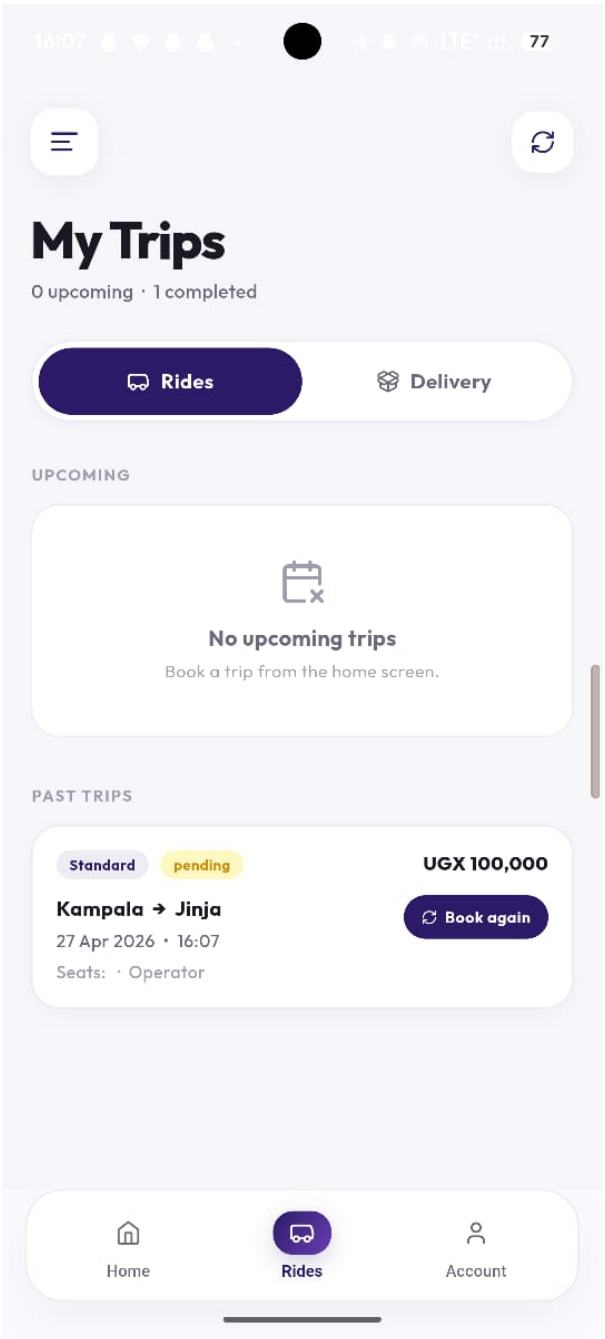
(a) Operator profile

(b) Company registration

Figure 5.4: Operator management interfaces

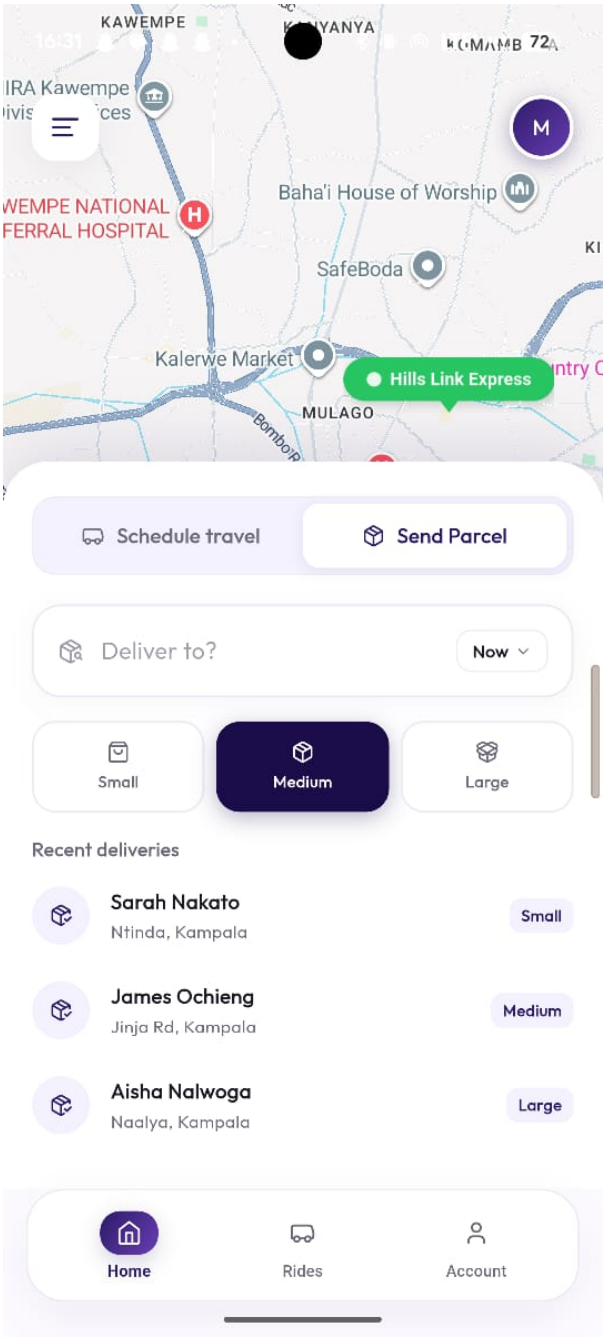


(a) Passenger dashboard

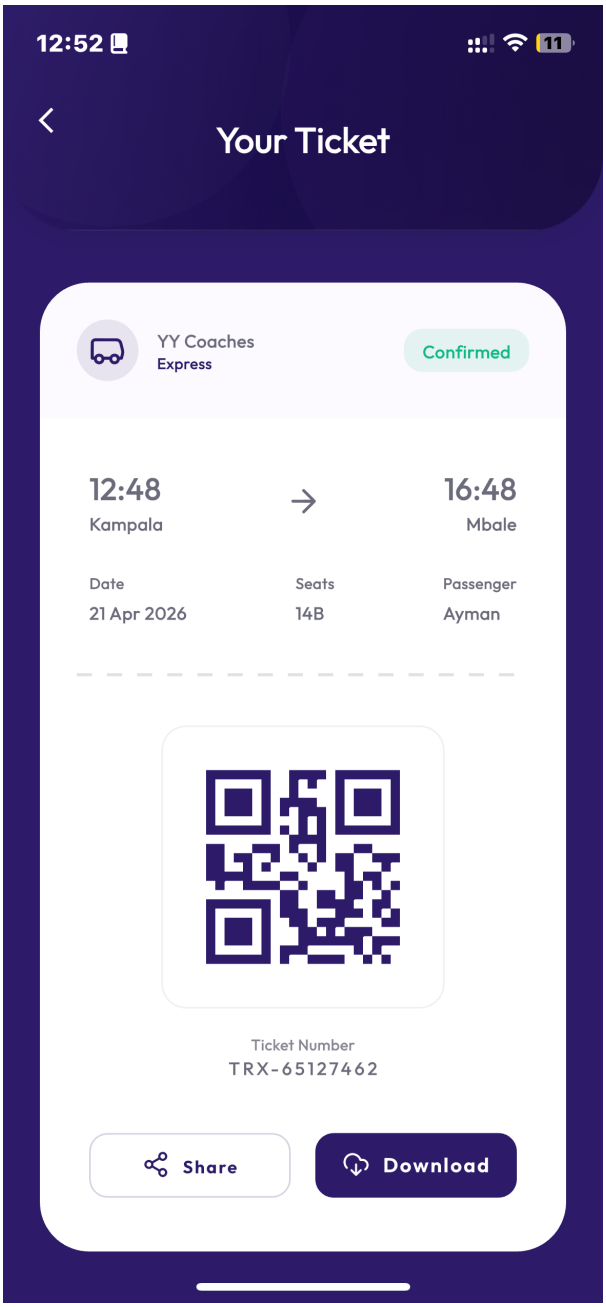


(b) Inputting route details

Figure 5.5: Passenger booking interfaces



(a) Parcel delivery details



(b) Digital ticket interface

Figure 5.6: Parcel and ticketing interfaces

5.2.4 Testing strategy and tools

The system underwent multiple layers of testing, covering individual components, integration points, overall system performance, and real-world usability:

Unit testing: Individual components were validated using unit tests to ensure that each module behaved as expected in isolation. For the backend, pytest was used to test Python and Django functionality, while the frontend Flutter applications employed flutter test and mockito for widget and logic testing. The development goal was to achieve greater than eighty five percent coverage on critical paths, ensuring that essential features functioned reliably under a variety of conditions.

Integration testing: Integration tests verified that different modules interacted correctly and that API contracts were honored. Backend APIs were tested using structured Postman collections to ensure correct request-response behavior, while end-to-end flows for mobile applications were tested using the Flutter Integration Driver. This ensured that data flowed seamlessly between clients, the backend, and external services.

System and Load Testing: To evaluate system performance under realistic usage conditions, load tests simulated up to 200 concurrent driver and passenger sessions using tools such as k6 or Locust. Performance metrics, including GPS update latency, were monitored to confirm that updates remained under 4 seconds even on 3G networks. These tests validated the system's scalability, responsiveness, and resilience under peak loads.

Real-World Field Testing: Controlled simulations alone were not sufficient to capture the practical challenges faced in private shuttle operations, particularly those identified during the interview with HillsLink Express. Instead, the system design and evaluation were guided by real-world operational scenarios derived from the interview findings.

These scenarios included common activities such as seat booking via phone, managing no-shows, handling mid-route passenger drop-offs, coordinating parcel deliveries, and responding to frequent customer inquiries about vehicle location and delivery status. By modelling these real-life interactions within the Traxis system, the study was able to assess how effectively the proposed solution addresses existing inefficiencies.

This approach ensured that the system was grounded in actual user needs and operational constraints, including limited digital literacy, reliance on phone communication, and the need for flexible booking and tracking mechanisms. As a result, Traxis was evaluated based on its ability to improve coordination, enhance transparency, and reduce operational burden within the context of private shuttle services.

User Acceptance Testing (UAT): Finally, the system underwent UAT(User Acceptance Tests) with a Hillslink passenger alongside the operator. Participants performed typical tasks using the system over a two-week period. Think-aloud sessions and standardized questionnaires such as the SUS(System Usability Scale) were employed to capture qualitative and quantitative feedback, validating that the system met user needs and expectations. The testing strategy leveraged a variety of professional tools, including Postman for API testing, Flutter DevTools for debugging and performance profiling, Firebase Test Lab for real-device testing, Sentry for crash reporting from the earliest stages of development, and Firebase Performance Monitoring for continuous perfor mance evaluation.

By employing this comprehensive, multi-layered testing and validation strategy, the Traxis system achieved high reliability, responsiveness, and usability, ensuring readiness for

real-world deployment and sustained user satisfaction

References

[1] Makerere University. (2026) Kampala at a crossroads: What new research reveals about mobility, governance, and the city's public health risks. Published January 8, 2026. [Online]. Available: https://www.mak.ac.ug/index.php/headlines/37215

[2] A. Rugamba, "Country scoping of research priorities on low-carbon transport in uganda," 2020, funded by DFID/UKAID under the Energy and Economic Growth Programme. [Online]. Available: https://transport-links.com/hvt-publications/country-scoping-of-research-priorities-on-low-carbon-transport-in-uganda

[3] Government of Uganda, Ministry of Finance, Planning and Economic Development, "Integrated transport infrastructure services annual monitoring report fy 2024/25," 2025, accessed 2026. [Online]. Available: https://www.finance.go.ug/sites/default/files/reports/Integrated%20Transport%20Infrastructure%20Services%20Annual%20Monitoring%20FY2024-25%20Report.pdf

[4] E. Masinde. (2024) Urban transportation sustainability in uganda: A critical analysis of kampala's transport challenges and climate-resilient solutions. Accessed via Scribd; document upload. [Online]. Available: https://www.scribd.com/document/938112103/Uganda-s-Transportation-Crisis

[5] ITD Services. (2026) Courier software vs manual management: A comparison. Accessed 2026-04-13. [Online]. Available: https://itdservices.in/Courier_Software_vs_Manual_Management_A_Comparison.php

[6] Uganda Communications Commission. (2025) New communications sector report highlights efforts to bridge access and usage gaps. Published 15 December 2025, accessed 2026-04-13. [Online]. Available: https://www.ucc.co.ug/new-communications-sector-report-highlights-efforts-to-bridge-access-and-usage-gaps/

[7] Codeware Limited. (2023) Benefits of implementing online bus seat booking system, pos and mobile application. Accessed 2026-04-13. [Online]. Available: https://vocal.media/journal/benefits-of-implementing-online-bus-seat-booking-system-pos-and-mobile-application

[8] A. H. Sayyed. (2024) The role of mobile apps in enhancing customer experience in 2024. LinkedIn Pulse article, accessed 2026-04-13. [Online]. Available: https://www.linkedin.com/pulse/role-mobile-apps-enhancing-customer-experience-2024-sayyed--stt1c

[9] I. Ndibatya and M. J. Booysen, "Minibus taxis in kampala's paratransit system: Operations, economics and efficiency," *Journal of Transport Geography*, vol. 88, p. 102853, 2020. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0966692320304415

[10] T. Courtright, E. Mbabazi, A. Oursler, and T. H. Tun. (2021) Reorganizing informal transport in uganda: Achieving a multimodality that works for all. Published October 14, 2021, accessed 2026-04-13. [Online]. Available: https://thecityfix.com/blog/reorganizing-informal-transport-in-uganda-achieving-a-multimodality-that-works-for-all/

[11] How we made it in Africa. (2023) Uganda: A tech solution for making public transport less 'hectic'. Published 8 February 2023, accessed 2026-04-13. [Online]. Available: https://www.howwemadeitinafrica.com/uganda-a-tech-solution-for-making-public-transport-less-hectic/151149/

[12] P. Manwaring and S. Wani, "Informal transport reform in kampala: Learning from cross-country experience," International Growth Centre (IGC), Tech. Rep., August 2021, policy Brief. [Online]. Available: https://www.theigc.org/sites/default/files/2021/09/Manwaring-et-al-August-2021-Policy-brief.pdf

[13] Entebbe Airport Shuttle. (2026) Private airport transfers. Accessed 2026-04-13. [Online]. Available: https://www.entebbeairportshuttle.com/private-airport-transfers/

[14] International Network for Transport and Accessibility in Low Income Communities (INTALInC), "Transport and social exclusion in uganda: Kampala case study report," VREF (Volvo Research and Educational Foundations), Tech. Rep., 2021, kampala case study report, accessed 2026-04-13. [Online]. Available: https://intalinc.org/wp-content/uploads/2021/03/kampala-vref_report.pdf

[15] D. Petersen and A. Behr, "The role of information technology in sustainable urban mobility development," *Research Square*, 2024. [Online]. Available: https://doi.org/10.21203/rs.3.rs-4351903/v1

[16] Y. Sun, C. Liu, and C. Zhang, "Mobile technology and studies on transport behavior: A literature analysis, integrated research model, and future research agenda," *Mobile Information Systems*, 2021. [Online]. Available: https://doi.org/10.1155/2021/9309904

[17] D. G. Rami Amin, "Affordable devices for all; innovative financing solutions and policy options to bride global digital divides," World Bank, Tech. Rep., 2024. [Online]. Available: https://documents1.worldbank.org/curated/en/099080723143031193/pdf/P1737510ac79240b90aaa10618d282c1780.pdf

[18] Mike, "Tecno, infinix, and itel: The secret behind their dominance in africa's smartphone market," 2025. [Online]. Available: https://tremhost.com/blog/tecno-infinix-and-itel-the-secret-behind-their-dominance-in-africas-smartphone-market/

[19] S. Hogarty, "7 best cheap phones that prove budget smartphones aren't rubbish," *The Independent (IndyBest)*, February 2025. [Online]. Available: https://www.independent.co.uk/extras/indybest/gadgets-tech/phones-accessories/best-cheap-phones-budget-b2505303.html

[20] H. Lanlehin, "Why are infinix and tecno phones so cheap? the real reasons explained," *TechCabal*, October 2025. [Online]. Available: https://techcabal.com/2025/10/20/why-are-infinix-and-tecno-phones-so-cheap-the-real-reasons-explained/

[21] R. E. Barka and I. Politis, "Driving into the future: A scoping review of smartwatch use for real-time driver monitoring," *Transportation Research Interdisciplinary Perspectives*, vol. 25, p. 101098, 2024. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2590198224000848

[22] C. L. Preciado-Ortiz, "Quality and use of mobile applications for transportation service: Influence on satisfaction," *MYN (Morelia, Mich.)*, 2021. [Online]. Available: https://www.redalyc.org/journal/5718/571867949003/

[23] Software Advice, "Gps vehicle tracking system — reviews, pricing & demo," https://www.softwareadvice.com.au/software/279003/gps-vehicle-tracking-system, 2025, fleet management software with real-time GPS tracking, driver behavior reports, and vehicle maintenance features.

[24] Y. Cao, Y. Tian, J. Tian, K. Liu, and Y. Wang, "Impact of built environment on residential online car-hailing trips: Based on mgwr model," *PLoS One*, vol. 17, 2022. [Online]. Available: https://www.ncbi.nlm.nih.gov/pmc/articles/PMC9671434/

[25] E. Ivanova, "Digitization of payments: Key to broader adoption of public transport," 2021. [Online]. Available: https://therecursive.com/digitization-of-payments-key-to-broader-adoption-of-public-transport-2/

[26] PHS, "Everything you need to know about cashless travelling," https://lexingham.com/everything-you-need-to-know-about-cashless-travelling/, 2024.

[27] S. M. Center and A. P. T. Association, "Shared mobility and the transformation of public transit," American Public Transportation Association, Tech. Rep., 2016. [Online]. Available: https://www.apta.com/wp-content/uploads/Resources/resources/reportsandpublications/Documents/APTA-Shared-Mobility.pdf

[28] Uber, "Uber payment options in uganda," 2025. [Online]. Available: https://www.uber.com/en-UG/blog/uber-payment-options-uganda/

[29] N. J. Kamanzi, "3 ways to topup your safeboda wallet," October 2019. [Online]. Available: https://medium.com/safeboda/3-ways-to-topup-your-safeboda-wallet-64c81a6176f8

[30] Bolt, "Alternative payment method offerings in bolt payments," 2025. [Online]. Available: https://help.bolt.com/add-ons/bolt-payments/

[31] Fact.MR, "Ride-hailing service market analysis by vehicle type (four wheeler, others (two wheeler, three wheeler)), by end-user type (personal, business), and region - global market insights 2023 to 2033," Fact.MR, Tech. Rep., 2023. [Online]. Available: https://www.factmr.com/report/ride-hailing-service-market

[32] K. Kukk, "The role of satellite communication in the creation of global data transmission networks," in *Proceedings of The 2nd International Conference on Satellite Communications*, vol. 4, 1996.

[33] D. Stopher, "Waze: The community-driven gps app revolutionizing navigation," 2025. [Online]. Available: https://nwconnected.co.uk/waze-the-community-driven-gps-app-revolutionizing-navigation/

[34] W. Team, "On route: How cellular connectivity keeps public transport running in real time," 2024. [Online]. Available: https://webbingsolutions.com/the-role-of-cellular-connectivity-in-modern-public-transport/

[35] K. Goto and Y. Kambayashi, "A new passenger support system for public transport using mobile database access," in *Proceedings of the 28th VLDB Conference*, 2002. [Online]. Available: https://www.vldb.org/conf/2002/S28P01.pdf

[36] Geotabteam, "The ultimate 2025 guide to fleet telematics: Maximize safety & efficiency," Tech. Rep., 2025 2025. [Online]. Available: https://www.geotab.com/blog/fleet-telematics-safety-efficiency-guide/

[37] A. Ghosh, "Mdm for fleet telematics: Vehicle management made smarter," Aug 2025. [Online]. Available: https://blog.scalefusion.com/mdm-for-fleet-telematics/#:~:text=Fleet%20telematics%20is%20essential%20for%20any%20organization,(OBD)%20and%20GPS%20is%20known%20as%20telematics.

[38] W. Kurniadi, "Digital transformation in the transportation and logistics industry," *Siber Journal of Transportation and Logistics*, vol. 3, pp. 1–6, 05 2025.

[39] Modeshift, "Cloud-based transit systems explained: What are they and how do they work?" *Modeshift Blog*, March 2024. [Online]. Available: https://www.modeshift.com/cloud-based-transit-systems-explained-what-are-they-and-how-do-they-work/

[40] RideWyze, "Why ride-hailing businesses need robust cloud-based solutions," *RideWyze Blog*. [Online]. Available: https://www.ridewyze.com/blog/why-ride-hailing-businesses-need-robust-cloud-based-solutions

[41] Lucidchart, "Reliability and availability in cloud computing," *Lucidchart Blog*. [Online]. Available: https://www.lucidchart.com/blog/reliability-availability-in-cloud-computing

[42] Nuvizzteam, "How real-time route planning boosts on-time deliveries & cuts costs in logistics," *Nuvizz Blog*, 2025. [Online]. Available: https://nuvizz.com/blog/real-time-route-planning-boosts-deliveries-cut-costs-in-logistics/

[43] RideWyzeTeam, "Automated ride scheduling: The future of dispatch software," *RideWyze Blog*, 2025. [Online]. Available: https://www.ridewyze.com/blog/automated-ride-scheduling-the-future-of-dispatch-software

[44] RideWyze, "Automated ride scheduling: The future of dispatch software," *RideWyze Blog*, accessed online. [Online]. Available: https://www.ridewyze.com/blog/automated-ride-scheduling-the-future-of-dispatch-software

[45] K. Waehner, "Dynamic pricing, real-time data streaming with apache kafka and flink," *Medium*. [Online]. Available: https://kai-waehner.medium.com/dynamic-pricing-real-time-data-streaming-with-apache-kafka-and-flink-1e7517055661

[46] Cloudfresh, "10 reasons why ci/cd is important for devops," *Cloudfresh Blog*. [Online]. Available: https://cloudfresh.com/en/blog/10-reasons-why-ci-cd-is-important-for-devops/

[47] Moiboo, "Vehicle diagnostics and maintenance in usa," *Moiboo Blog*. [Online]. Available: https://www.moiboo.com/blog/usa-united-states/vehicle-diagnostics-and-maintenance-in-usa/

[48] M. Amola, "Cloud computing in enhancing cyber security for businesses," 03 2025. [Online]. Available: https://www.researchgate.net/publication/389879495_Cloud_Computing_in_Enhancing_Cyber_security_for_Businesses

[49] Google Cloud, "Gdpr and google cloud," 2025. [Online]. Available: https://cloud.google.com/privacy/gdpr

[50] Uber Technologies Inc. (2026) About us. Accessed 2026-04-13. [Online]. Available: https://www.uber.com/us/en/about/

[51] ——. (2026) Uber car rental faq. Accessed 2026-04-13. [Online]. Available: https://help.uber.com/driving-and-delivering/article/uber-car-rental-faq?nodeId=32cbf39c-44e7-4cee-a99f-bb97382089a2

[52] ——. (2026) How does uber work? Accessed 2026-04-13. [Online]. Available: https://www.uber.com/us/en/about/how-does-uber-work/

[53] R. Kayi, "Effects of uber transport to tourism development in kampala city," Master's thesis, Makerere University, Kampala, Uganda, 2019. [Online]. Available: https://dissertations.mak.ac.ug/handle/20.500.12281/6381

[54] D. Isaac, "Impact of uber transportations on the informal public transport operation in uganda," unpublished manuscript. [Online]. Available: IMPACT_OF_UBER_TRANSPORTATIONS_ON_THE_INFORMAL_PUBLIC_TRANSPORT_OPERATION_IN_UGANDA_By_DESIRE_ISAAC_0700193521_rtf

[55] M. Staff, "Uber, online taxi firm launched in kampala," *Daily Monitor (Uganda)*, June 2016, accessed: 2025-11-18. [Online]. Available: https://www.monitor.co.ug/uganda/news/national/uber-online-taxi-firm-launched-in-kampala-1652628

[56] B. Musinguzi and T. Courtright. (2022) Uganda's safeboda has upset its riders and disappointed its customers. Published March 24, 2022, accessed 2026-04-13. [Online]. Available: https://restofworld.org/2022/ugandas-safeboda-has-upset-its-riders-and-disappointed-its-customers/

[57] J. Muhindo. (2025) Rewriting the ride culture in kampala. Published May 16, 2025, accessed 2026-04-13. [Online]. Available: https://www.safeboda.com/safeboda-blog/rewriting-the-ride-culture-in-kampala

[58] SafeBoda. (2026) Frequently asked questions (uganda). Accessed 2026-04-13. [Online]. Available: https://www.safeboda.com/uganda-website/faqs-nbo

[59] ——. (2024) Safecar drivers online training: Accepting rides. Driver training module, accessed 2026-04-13. [Online]. Available: https://www.safeboda.com/drivers-online-training/safecar/kla/en/course-page

[60] N. AlQatami. (2022) Can digital uptake help transform societies in ldcs? how safeboda is championing reliable delivery and transport services in kampala. Published May 30, 2022, accessed 2026-04-13. [Online]. Available: https://www.uncdf.org/article/7731/safeboda-may2022

[61] Dignited, "Uber vs taxify vs safeboda uganda: Price and feature comparison," 2018. [Online]. Available: https://www.dignited.com/26966/uber-vs-taxify-vs-vs-safeboda-uganda-prices-and-feature-comparison/

[62] S. Saddier, "Are motorcycle taxis competing with collective public transport? analyzing the role of boda-bodas in kampala's urban mobility system," Tech. Rep., 2025. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0739885925000459

[63] D. Katushabe, "Safecar by safeboda becomes the most preferred car-hailing service for safety and convenience," *SafeBoda*, 2025. [Online]. Available: https://www.safeboda.com/safeboda-blog/safecar-by-safeboda-becomes-the-most-preferred-car-hailing-service-for-safety-and-convenience

[64] Trend News Agency. (2018) Taxify says central asia a "very interesting" region. Published 1 August 2018, accessed 2026-04-14. [Online]. Available: https://www.trend.az/business/economy/2935990.html

[65] aPurple, "Bolt business model," Jun 2025. [Online]. Available: https://www.apurple.co/bolt-business-model/

[66] Q. Africa, "Uber rival taxify changes name to bolt, adds scooters," 2019. [Online]. Available: https://qz.com/africa/1568416/uber-rival-taxify-changes-name-to-bolt-adds-scooters?utm_medium=sharefromsite&utm_source=quartz_link

[67] P. T. Magazine, "Ride-sharing service taxify officially launches in uganda," https://pctechmag.com/2017/10/ride-sharing-service-taxify-officially-launches-in-uganda/, Oct 2017.

[68] J. Salmon, "Driving or ride-hailing: Which one is cheaper?" *Daily Monitor (Uganda)*, 2023. [Online]. Available: https://www.monitor.co.ug/uganda/business/prosper/driving-or-ride-hailing-which-one-is-cheaper--4299538

[69] A. Shoaib. (2021) Meet lyft: "uber's twin" in the usa and canada. Published February 19, 2021, accessed 2026-04-14. [Online]. Available: https://www.relawding.com/meet-lyft-ubers-twin-in-the-usa-and-canada/

[70] Yahoo News. (2022) Lyft brings shared rides back. Published May 5, 2022, accessed 2026-04-14. [Online]. Available: https://news.yahoo.com/lyft-brings-shared-rides-back-021404116.html

[71] Lyft Inc. (2026) Rider policies for lyft rides. Accessed 2026-04-14. [Online]. Available: https://help.lyft.com/hc/en-ca/all/articles/115013080388-Rider-policies-for-Lyft-rides

[72] PYMNTS. (2020) Lyft turns to meal, grocery deliveries. Published April 16, 2020, accessed 2026-04-14. [Online]. Available: https://www.pymnts.com/news/ridesharing/2020/lyft-turns-to-meal-grocery-deliveries/

[73] The Rideshare Guy. (2023) Lyft delivery driver: How it works and what to expect. Accessed 2026-04-14. [Online]. Available: https://therideshareguy.com/lyft-delivery-driver/

[74] A. J. Hawkins. (2020) Lyft launches essential deliveries—but it's not an uber eats competitor. Published April 15, 2020, accessed 2026-04-14. [Online]. Available: https://www.fastcompany.com/90491483/lyft-launches-essential-deliveries-but-its-not-an-uber-eats-competitor

[75] J. Littman. (2021) Lyft, olo partner to offer restaurant delivery. Published December 14, 2021, accessed 2026-04-14. [Online]. Available: https://www.restaurantdive.com/news/lyft-olo-partner-to-offer-restaurant-delivery/611486/

[76] J. Wiklund. (2021) How is lyft's food-delivery strategy different? Published December 21, 2021, accessed 2026-04-14. [Online]. Available: https://foodinstitute.com/focus/how-is-lyfts-food-delivery-strategy-different/

[77] Moovit Inc. (2026) About moovit: The leading maas solutions provider. Accessed 2026-04-14. [Online]. Available: https://moovit.com/about-us/

[78] ——. (2026) Mastering your commute: A guide to navigating with the moovit app. Published 21 January 2026, accessed 2026-04-14. [Online]. Available: https://moovit.com/blog/navigating-with-moovit/

[79] TransitApp, "Transit app: Real-time public transport and micromobility planning," 2025. [Online]. Available: https://transitapp.com/vision

[80] Transit App. (2026) How to use transit. Accessed 2026-04-14. [Online]. Available: https://help.transitapp.com/article/93-how-to-use-transit

[81] Swvl Holdings Corp, "Transforming urban transit: How swvl tackles mea's biggest mobility challenges," August 2025. [Online]. Available: https://www.swvl.com/blog/transforming-urban-transit-how-swvl-tackles-meas-biggest-mobility-challenges

[82] S. Narain. (2025) Transforming urban transit: How swvl tackles mea's biggest mobility challenges. Published August 22, 2025, accessed 2026-04-14. [Online]. Available: https://www.swvl.com/blog/transforming-urban-transit-how-swvl-tackles-meas-biggest-mobility-challenges

[83] Via Transportation, Inc., "Via's vision for smart demand-responsive transport in ireland," 2025. [Online]. Available: https://ridewithvia.com/ireland

[84] Business Model Canvas Template. (2026) Via: How it works. Accessed 2026-04-14. [Online]. Available: https://businessmodelcanvastemplate.com/blogs/how-it-works/via-how-it-works

[85] Z. Wu, "The analysis of grab's business model and revenue strategies," *Advances in Economics Management and Political Sciences*, vol. 145, no. 1, pp. 21–29, 2025. [Online]. Available: https://www.researchgate.net/publication/387716884_The_Analysis_of_Grab%27s_Business_Model_and_Revenue_Strategies

[86] Jugnoo. (2022) How grab works: The on-demand business model. Accessed 2026-04-14. [Online]. Available: https://jugnoo.io/how-grab-works-the-on-demand-business-model/

Appendices

Appendix A : Budget

The research and development of the Traxis mobile application will be conducted over a period of 3 months.

Item	**Description**	**Estimated Cost (UGX)**
Internet and Communication	Research coordination, data collection, and documentation	100,000
Software Tools	Firebase, Google Maps API, and development plugins	200,000
Cloud Hosting	Server and database hosting for testing phase	100,000
Miscellaneous	Contingency and administrative expenses	100,000
Total		**500,000**

Table 1: Estimated Project Budget

Appendix B: Event Scheduling (Workplan)

This appendix presents the project workplan outlining the planned activities for the development of the Traxis system. The workplan is organized chronologically and aligns with the Agile-inspired methodology described in Chapter 4. It defines the scheduled development events, their durations, and intended outputs, providing a structured roadmap for the successful execution of the project.

Month 1: Foundation Phase (January 2026)

Activity 1: Backend and Database Setup
Scheduled Period: 1 January 2026 – 14 January 2026
During this period, the project team will establish the foundational backend infrastructure for the system. This will include configuring the Django backend framework, setting up the PostgreSQL database with PostGIS extensions, defining core data models, and creating initial RESTful API endpoints. The expected outcome will be a functional backend environment capable of supporting authentication, data persistence, and future feature development.

Activity 2: Authentication and Location Services Implementation
Scheduled Period: 15 January 2026 – 31 January 2026
This activity will focus on implementing secure authentication mechanisms and geolocation services. Firebase Authentication will be integrated to support phone-number-based user login, while Google Maps APIs will be configured for real-time location tracking and routing. Backend logic for managing user sessions and location updates will be implemented to support subsequent ride- matching functionality.

Month 2: Core Interaction Phase (February 2026)

Activity 3: User Interface and Authentication Workflow Development
Scheduled Period: 1 February 2026 – 14 February 2026
During this phase, the client-side applications will be developed using Flutter. User interface components, navigation flows, and authentication workflows for both passengers and drivers will be implemented from a shared codebase. The primary objective will be to deliver a functional, responsive, and user-friendly mobile interface capable of interacting securely with backend services.

Activity 4: Ride Management Logic and AI Integration
Scheduled Period: 15 February 2026 – 28 February 2026
This activity will involve implementing the core ride management features of the system. Ride request handling, driver availability broadcasting, and passenger–driver matching logic will be developed within the backend. AI services will be integrated through MCP to support predictive matching and demand-aware allocation. Real-time notifications will be enabled using Firebase Cloud Messaging to ensure timely communication between system components.

Month 3: System Enhancement and Finalization Phase (March 2026)

Activity 5: Analytics and Administrative Tools Development
Scheduled Period: 1 March 2026 – 14 March 2026
This phase will focus on the development of administrative and analytical capabilities. The Admin Portal will be implemented using Vue.js to provide tools for user management, trip monitoring, and system analytics. Supporting backend endpoints for reporting, logging, and administrative operations will be completed to enable effective system oversight.

Activity 6: Performance Optimization, Accessibility, and System Validation
Scheduled Period: 15 March 2026 – 31 March 2026
The final activity will concentrate on system refinement and deployment readiness. Performance optimization will be conducted across the backend and mobile applications, including database tuning and background task optimization. Accessibility enhancements will be implemented to improve usability for diverse user groups. Comprehensive testing, bug fixing, and final system validation will be carried out to ensure compliance with project requirements.

Workplan Summary

The proposed workplan will ensure a phased and systematic approach to system development, progressing from foundational setup to core functionality and final optimization. By adhering to clearly defined activities and timelines, the project will maintain effective coordination, reduce development risks, and support the timely delivery of the Traxis system in accordance with the proposed methodology.